



Créer et intégrer un Chatbot multimodal dans une application web - partie 1

Table des matières

I - Paramétrage du backend et création du chatbot (texte)	3
A. Paramétrage du backend : FastAPI + dépendances IA.....	3
B. Création du chatbot (sans IA) v1.1.....	12
C. Chatbot sans IA v.1.2	22
D. Création du chatbot (texte avec IA).....	28
II - Refactorisation du backend	36
A. Comment passer d'une démo à un modèle scalable ?	36
B. Comment connecter le backend au frontend ?	40
C. Amélioration data set et création mode DEMO	54
III - Synthèse	65
IV - Auto-évaluation	67
A. Exercice : Quiz	67
B. Exercice : Cas pratique.....	71
Solutions des exercices	72

I Paramétrage du backend et création du chatbot (texte)

A. Paramétrage du backend : FastAPI + dépendances IA

La réussite du projet « Bon Vivant » repose sur un backend robuste, bien documenté et simple à prendre en main.

Ces 3 visions ont guidé la stack technique du projet.

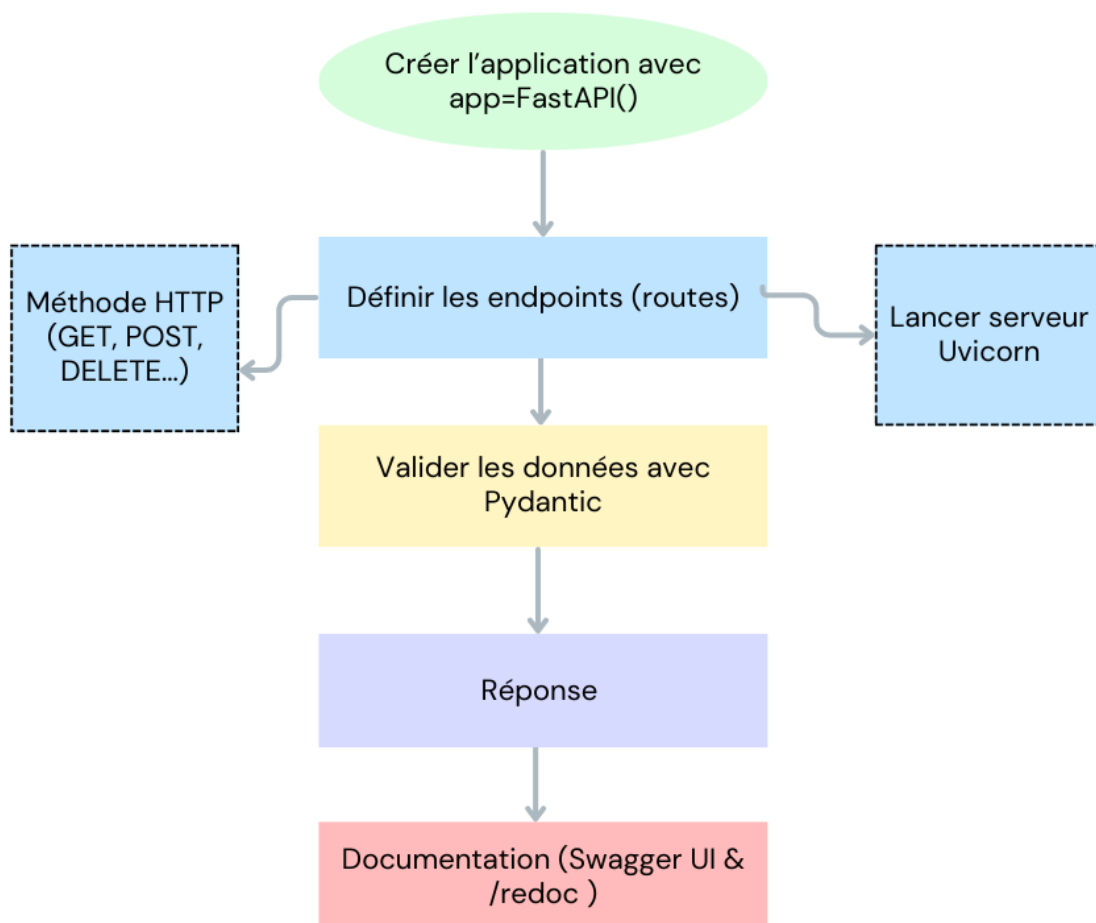
En effet, le cœur de ce projet est géré par FastAPI.

Az Définition

FastAPI est un framework qui permet de créer des applications web très rapides en Python. Il a été créé en 2018 par Sebastián Ramírez. Sebastián avait pour objectif de rendre la création d'API simple et rapide tout en bénéficiant des fonctionnalités modernes de Python, à savoir la validation automatique des données ainsi que la gestion asynchrone. De nos jours, FastAPI est devenu l'un des frameworks Python les plus populaires et ses performances sont proches de quelques-uns des frameworks les plus connus du marché tels que Node.js par exemple.

Comment créer une application web avec FastAPI ?

Méthode



Créer une application web avec FastAPI repose sur 7 éléments clés.

1. On commence par créer l'application avec "app=FastAPI()".

Quand on écrit app= FastAPI(), on crée une instance de notre application.

Cette instance contiendra nos endpoints (routes), nos configurations (sécurité, CORS...) et enfin les middlewares de l'application (gestion des logs, ou encore de l'authentification).

```
app = FastAPI()
```

Cet objet app sera ensuite transmis au serveur ASGI (Uvicorn) afin de répondre aux requêtes utilisateurs. Nous reviendrons sur Uvicorn dans les prochaines lignes.

2. Deuxièmement, abordons la question des endpoints.

Un endpoint, aussi appelé route est une URL précise que le client peut appeler afin d'interagir avec l'application. Par exemple, si notre API a un endpoint /bonjour, nous pourrions y accéder via notre navigateur ou avec un appel HTTP.

Chaque endpoint correspond à une fonction Python.

```
# Endpoint simple (GET)
@app.get("/bonjour")
def dis_bonjour():
    return {"message": "Bonjour !!"}
```

```
1 @app.get("/bonjour")
2 def dis_bonjour():
3     return {"message": "Bonjour"}
```

Méthode

Ici, @app.get("/bonjour") signifie que lorsqu'une personne fait une requête GET sur /bonjour, FastAPI exécute la fonction dis_bonjour.

De plus, dans FastAPI une fonction qui sert d'endpoint peut être synchrone (def) ou asynchrone (async def)

```
def
```

```
async def
```

3. Troisième élément principal : les méthodes HTTP.

Les méthodes HTTP sont les verbes qui définissent l'action à effectuer. Les voici :

- GET permet de lire ou de récupérer des données.
- POST permet de créer de nouvelles données.
- PUT permet de remplacer complètement des données existantes.
- PATCH permet de modifier partiellement des données.

Retrouvez des exemples des autres méthodes dans la documentation FastAPI officielle First Steps¹.

4. Quatrième étape : la validation des données.

Dans le cadre de FastAPI, la validation des données est gérée par Pydantic.

En effet, quand un utilisateur envoie des données à notre API, par exemple une adresse e-mail, il est important de vérifier que les données sont correctes avant de les utiliser.

C'est la vocation de Pydantic. Grâce à Pydantic, nous pouvons définir un modèle de données qui décrit exactement ce que l'on attend comme données.

```
# Validation Pydantic
class User(BaseModel):
    name: str
    age: int
    email: EmailStr
```

```
1 class User(BaseModel):
2     name : str
3     email: EmailStr
```

EmailStr est un type spécial fourni par Pydantic. Celui-ci vérifie que la valeur reçue est bien un email valide. Ainsi, si un utilisateur envoie un email mal formaté, FastAPI renverra automatiquement une erreur. Cela permet de gagner du temps et d'éviter les bugs liés aux données mal formatées.

Il existe d'autres types. EmailStr est l'un d'eux. La liste complète se trouve dans la documentation officielle de Pydantic : Pydantic²

5. Ensuite vient la réponse.

Par défaut, FastAPI renvoie du JSON mais l'on peut aussi retourner du HTML, des fichiers à télécharger ou encore des images.

Afin de garantir un format de données constamment cohérent, il est également possible de structurer les données avec Pydantic.

1. <https://fastapi.tiangolo.com/tutorial/first-steps/#path>

2. <https://docs.pydantic.dev/latest/>

```

from fastapi import FastAPI
from pydantic import BaseModel, EmailStr

app = FastAPI()

# Endpoint simple (GET)
@app.get("/bonjour")
def dis_bonjour():
    return {"message": "Bonjour !"}

# Validation Pydantic
class User(BaseModel):
    name: str
    age: int
    email: EmailStr

# Endpoint (POST) avec validation automatique
@app.post("/users")
def create_user(user: User):
    return {
        "message": "L'utilisateur a été créé avec succès",
        "data": user
    }

```

```

1 from fastapi import FastAPI
2 from pydantic import BaseModel, EmailStr
3
4 app = FastAPI()
5
6 # Endpoint simple (GET)
7 @app.get("/bonjour")
8 def dis_bonjour():
9     return {"message": "Bonjour !"}
10
11 # Validation Pydantic
12 class User(BaseModel):
13     name: str
14     age: int
15     email: EmailStr
16
17 # Endpoint (POST) avec validation automatique
18 @app.post("/users")
19 def create_user(user: User):
20     return {
21         "message": "L'utilisateur a été créé avec succès",
22         "data": user
23 }

```

Ainsi, si le client envoie une donnée conforme, FastAPI accepte et retourne ces données formatées.

```
INFO: 127.0.0.1:53430 - "POST /users HTTP/1.1" 200 OK
```

```
0/users" \
-H "Content-Type: application/json" \
-d '{"name":"Alice","age":30,"email":"alice@example.com"}'
{"message":"Utilisateur créé avec succès","data":{"name":"Alice","age":30,"email":"alice@example.com"}}
```

```
1 {
2   "name": "Alice",
3   "age": 30,
4   "email": "alice@example.com"
5 }
```

Par contre, si le client envoie une donnée non conforme, FastAPI renvoie automatiquement une erreur 422 ainsi qu'un message expliquant que l'email doit être sous un format valide.

INFO: 127.0.0.1:53419 - "POST /users HTTP/1.1" 422 Unprocessable Entity

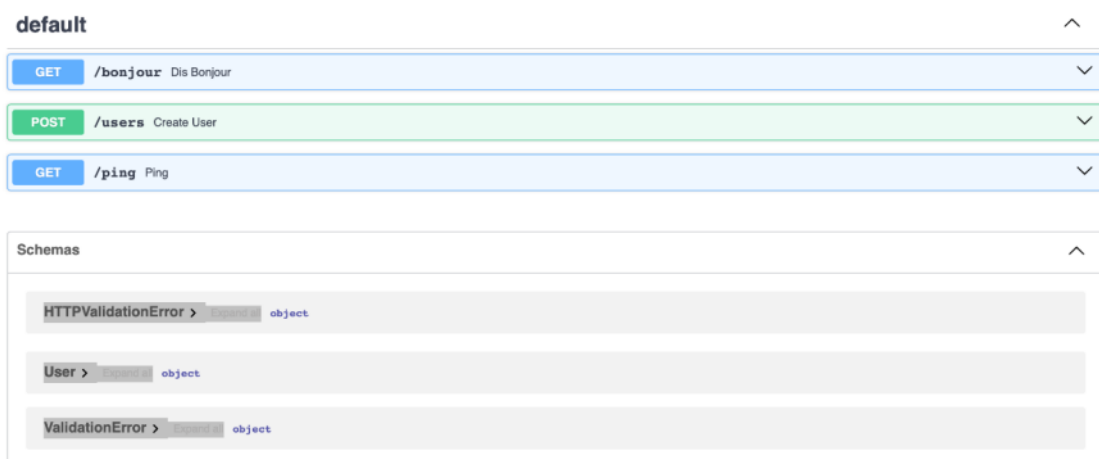
```
-H "Content-Type: application/json" \
-d '{"name":"Jean","age":"trente","email":"email.fr"}'
{"detail":[{"type":"int_parsing","loc":["body","age"],"msg":"Input should be a valid integer, unable to parse string as an integer","input":"trente"}, {"type":"value_error","loc":["body","email"],"msg":"value is not a valid email address: An email address must have an @-sign.","input":"email.fr"}]}
```

```
1 {
2   "name": "Jean",
3   "age": "trente",
4   "email": "email.fr"
5 }
```

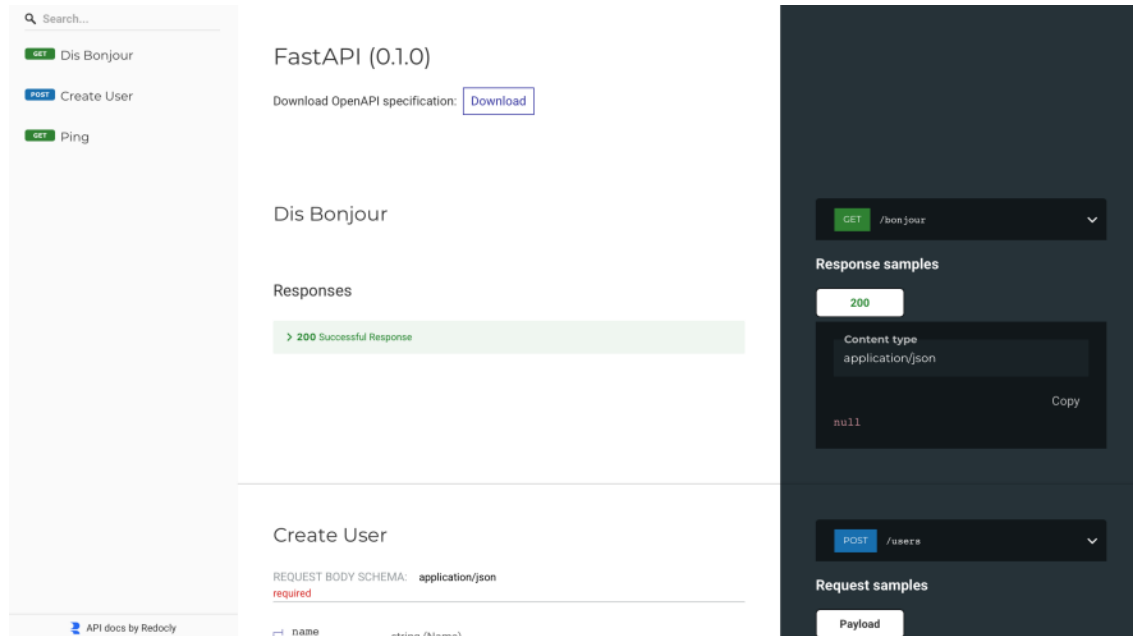
6. Une des particularités de FastAPI est que la documentation est générée automatiquement. Celle-ci est accessible depuis le navigateur et de deux manières.

Commençons par /docs. Depuis votre localhost/docs (exemple : <http://localhost:8000/docs>) vous pouvez accéder à /docs via Swagger UI. Cela nous permet d'effectuer des tests en direct et envoyer des requêtes POST/GET, ajouter un body JSON, voir les réponses etc. C'est très pratique pour déboguer l'API.

FastAPI 0.1.0 OAS 3.1
/openapi.json



Deuxièmement, il y a /redoc. Redoc permet d'accéder à une documentation descriptive basée sur Redoc. Contrairement à /docs, la documentation est moins interactive, mais elle est aussi plus lisible et structurée. Cette documentation est pensée pour être partagée à des clients, des équipes produit ou des personnes non-techniques qui aimeraient comprendre ce que fait l'API. Pour accéder à Redoc, ajoutez /redoc à votre localhost exemple : <http://localhost:8000/redoc>



7. Septième point clé : ASGI et Uvicorn.

Ce qui rend FastAPI si performant, c'est aussi que FastAPI est ce qu'on appelle une application ASGI. C'est-à-dire qu'il suit une norme qui définit comment une application Python peut communiquer avec un serveur web.

Un ASGI (Asynchronous Server Gateway Interface) est une spécification technique : c'est le langage commun entre notre application et le serveur. Très concrètement, FastAPI ne parle pas directement au navigateur ou au frontend : il a besoin d'un intermédiaire, et c'est le rôle du serveur ASGI (ici Uvicorn).

Avant ASGI, on utilisait surtout WSGI (Web Server Gateway Interface), par exemple avec Flask ou Django. Mais WSGI avait une limite : il ne permettait que du fonctionnement synchrone. Une requête devait être traitée entièrement avant de passer à la suivante.

ASGI apporte :

- De l'asynchronisme natif (gérer plusieurs connexions en parallèle)
- De la polyvalence (support du http et autres protocoles)
- Et surtout une bien meilleure scalabilité et rapidité

C'est idéal pour l'IA, car une requête peut parfois durer plusieurs secondes (par exemple le temps qu'un modèle génère une réponse). ASGI permet de gérer tout cela sans bloquer le reste de l'application.

Terminons par notre serveur Uvicorn.

Uvicorn a pour but :

- D'écouter les demandes qui arrivent sur internet
- De les transmettre à FastAPI *via* le protocole ASGI
- De gérer la concurrence (c'est-à-dire plusieurs requêtes en même temps).
- De s'occuper du démarrage et de l'arrêt de l'application.
- D'ajouter des middlewares comme les logs, la sécurité ou la gestion des accès depuis d'autres sites (CORS)
- Et en mode développement, il permet de recharger automatiquement le code avec - - reload.

```
demo@dev uvicorn api.main:app --reload
```



```
1 uvicorn api.main:app --reload
```

Par ailleurs, les commandes de base que nous utiliserons pour Uvicorn sont :

- `uvicorn api.main:app --reload` : pour lancer le serveur. Ici, nous indiquons à uvicorn de lancer l'application FastAPI via l'objet `app` contenu dans le fichier `main`, lui-même situé dans le dossier `api`.

L'option `--reload` permet de redémarrer automatiquement le serveur à chaque modification du code.

- `CTRL + C` : pour stopper le serveur depuis le terminal.

Retrouvez l'ensemble des commandes d'Uvicorn dans la documentation officielle : Uvicorn³

Vous pouvez accéder à la documentation officielle de FastAPI sur le site officiel de Sebastián Ramírez : fastapi⁴

Ceci étant fait, nous pouvons passer à la suite du cours.

Mais avant cela, définissons quelques termes :

Quelques termes à définir avant de poursuivre

Az Définition

Tout au long du cours nous emploierons plusieurs termes.

Tout d'abord, LLama est le modèle IA local que nous utiliserons à la fois pour la génération de texte, et d'image.

Quant à lui, le terme LLM local signifie que l'on va télécharger un modèle IA directement sur notre machine afin d'éviter de faire appel à une API.

VOSK est le modèle IA choisi pour faire de la reconnaissance vocale locale open source.

ML Flow et Github Actions seront présents pour garantir la robustesse de notre application via le versionning IA et l'automatisation de tests.

Enfin, nous parlerons d'environnement virtuel (`.venv`). Ici, nous allons isoler toutes les dépendances Python que nous allons installer pour la création de ce projet afin d'éviter qu'il y ait des conflits entre les versions de librairies, d'un projet à un autre.

Commençons l'installation de nos différentes dépendances.

Rendez-vous dans le backend de votre projet puis tapez les commandes suivantes une par une.

```
1 cd backend
```

```
demo@mac cd backend
demo@mac
```

```
1 python3 -m venv .venv
```

```
demo@mac python3 -m venv .venv
demo@mac
```

```
1 source .venv/bin/activate
```

```
demo@mac source .venv/bin/activate
(.venv) demo@mac
```

3. <https://www.uvicorn.org/#command-line-options>

4. <https://fastapi.tiangolo.com/>

Sous windows :

```
1 cd backend
2 python -m venv .venv
3 .\.venv\Scripts\activate
```

Ensuite, il y aura quelques particularités en fonction de votre OS. Suivez les recommandations adaptées à votre OS.

Pour Mac Apple Silicon (M1, M2, M3 etc), dans le terminal et depuis le dossier backend, venv activé, vous allez installer llama-cpp-python avec support Metal. Cela permettra d'améliorer les performances de notre IA locale.

```
(.venv) demo@mac CMAKE_ARGS="-DLLAMA_METAL=on" FORCE_CMAKE=1 pip install 'llama-cpp-python[server]'
```

```
1 CMAKE_ARGS="-DLLAMA_METAL=on" FORCE_CMAKE=1 pip install 'llama-cpp-python[server]'
```

Ensuite, vous pouvez vérifier que l'installation a bien été effectuée grâce aux commandes suivantes.

```
(.venv) demo@mac pip show llama-cpp-python
Name: llama_cpp_python
Version: 0.3.16
Summary: Python bindings for the llama.cpp library
Home-page:
Author:
Author-email: Andrei Betlen <abetlen@gmail.com>
License: MIT
Location: /Users/beverllydorval/Documents/tuto-bonvivant-project-demo/backend/.venv/lib/python3.12/site-packages
Requires: diskcache, jinja2, numpy, typing-extensions
Required-by:
(.venv) demo@mac
```

```
(.venv) demo@mac python -c "import llama_cpp; print(llama_cpp.__version__)"
0.3.16
(.venv) demo@mac
```

```
1 pip show llama-cpp-python
2 python -c "import llama_cpp; print(llama_cpp.__version__)"
```

Nous allons ensuite installer les autres dépendances nécessaires à notre projet.

Tout d'abord, FastAPI. Comme nous l'avons vu, il s'agit d'un framework Python qui nous permettra de créer notre application web.

Deuxièmement, Uvicorn qui est le serveur ASGI qui fera tourner FastAPI.

Troisièmement, nous aurons Vosk. Vosk est l'IA de reconnaissance vocale que nous allons utiliser dans le cadre de la fonction voix du chatbot.

Il y aura également, pyttsx3. Il s'agit d'une synthèse vocale pour que le chatbot lise ses recommandations à voix haute.

Transformers, que nous avons déjà évoqué dans ce cours, est l'outil de NLP de HuggingFace. Nous utiliserons principalement Transformers afin de créer notre endpoint d'analyse de sentiment.

Nous téléchargeons vaderSentiment qui nous permettra de tester l'analyse de sentiment de manière simplifiée.

Avant-dernière dépendance, il s'agit de pydantic[dotenv] qui, comme nous l'avons évoqué précédemment, sera particulièrement utile à l'étape de validation et structuration des données.

Et enfin, MLflow qui nous sera utile pour la gestion du versionnage et suivi de nos modèles IA.

```
(.venv) demo@mac pip install fastapi uvicorn vosk pyttsx3 transformers vaderSentiment 'pydantic[dotenv]' mlflow
```

```
1 cd backend
2 pip install fastapi uvicorn vosk pyttsx3 transformers vaderSentiment
'pydantic[dotenv]' mlflow
```

Pour Windows ou Linux (Intel/AMD, pas Metal)

Si vous êtes sous Windows ou Linux, tapez la commande suivante dans votre terminal depuis le dossier backend et en ayant votre venv d'activé.

```
1 cd backend
2 pip install fastapi uvicorn llama-cpp-python[server] vosk pytttsx3 transformers
  vaderSentiment 'pydantic[dotenv]' mlflow
```

Enfin, nous allons créer le fichier main.py. Celui-ci servira à orchestrer tous les modules IA et fera l'interface entre le front et le back.

```
(.venv) demo@mac cd api
```

```
(.venv) demo@mac touch main.py
```

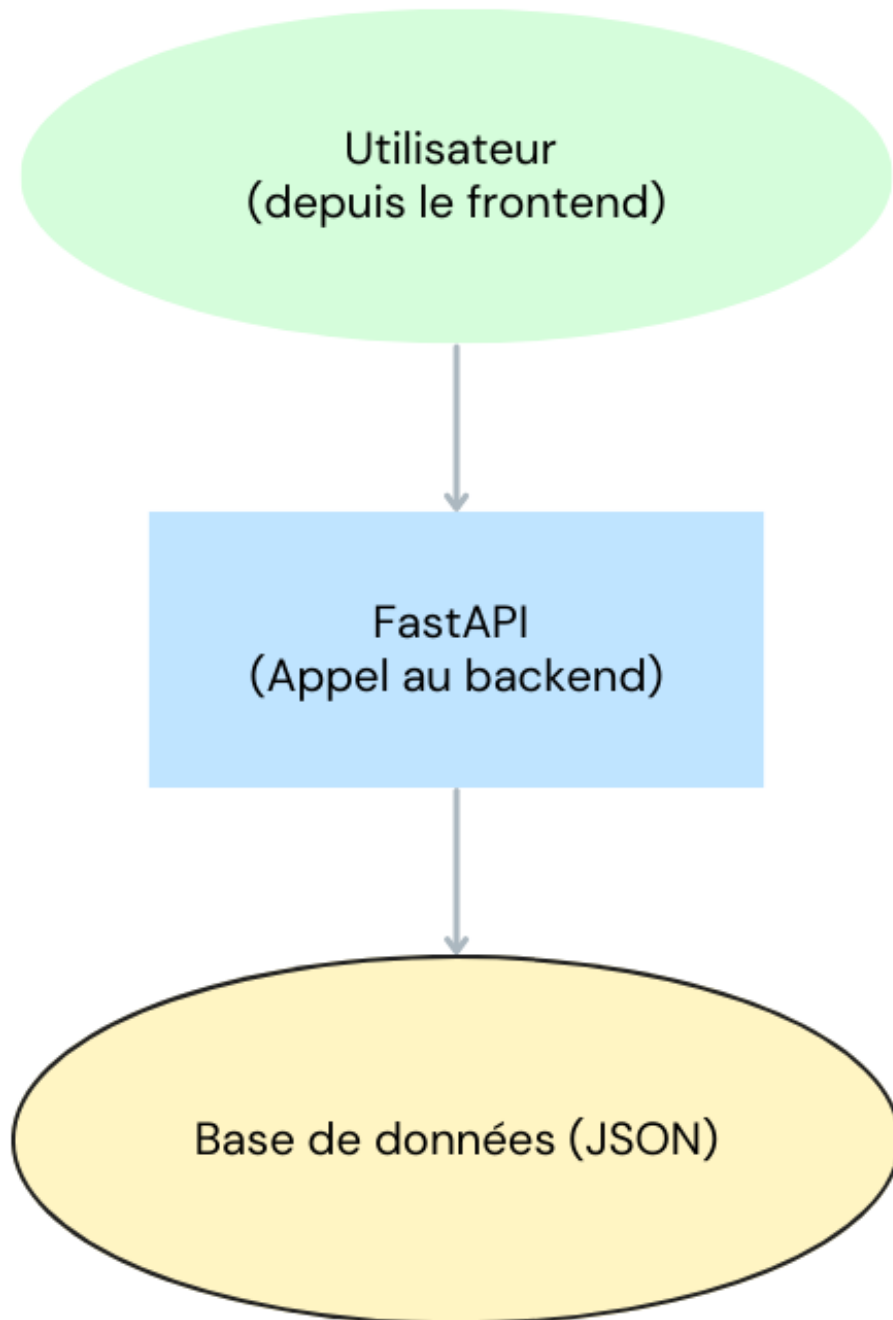
```
1 cd api
2 touch main.py
```

Vous venez de terminer l'initialisation de votre backend. Nous sommes maintenant prêts à coder nos premiers endpoints.

B. Création du chatbot (sans IA) v1.1

Avant d'intégrer l'IA, on commence par une version très simple de notre chatbot. L'objectif ici va être de valider le pipeline technique sans complexité.

Voici le flux de cette première version du chatbot :



L'utilisateur fait une requête depuis le frontend, cette requête fait appel au backend et donc à FastAPI. Le backend cherche une réponse grâce à notre base de données JSON et retourne une réponse.

Pour commencer, nous allons créer une branche dev. On travaille toujours sur une branche dev afin d'éviter de casser la version principale du projet.

À la racine du projet, tapez le code suivant.

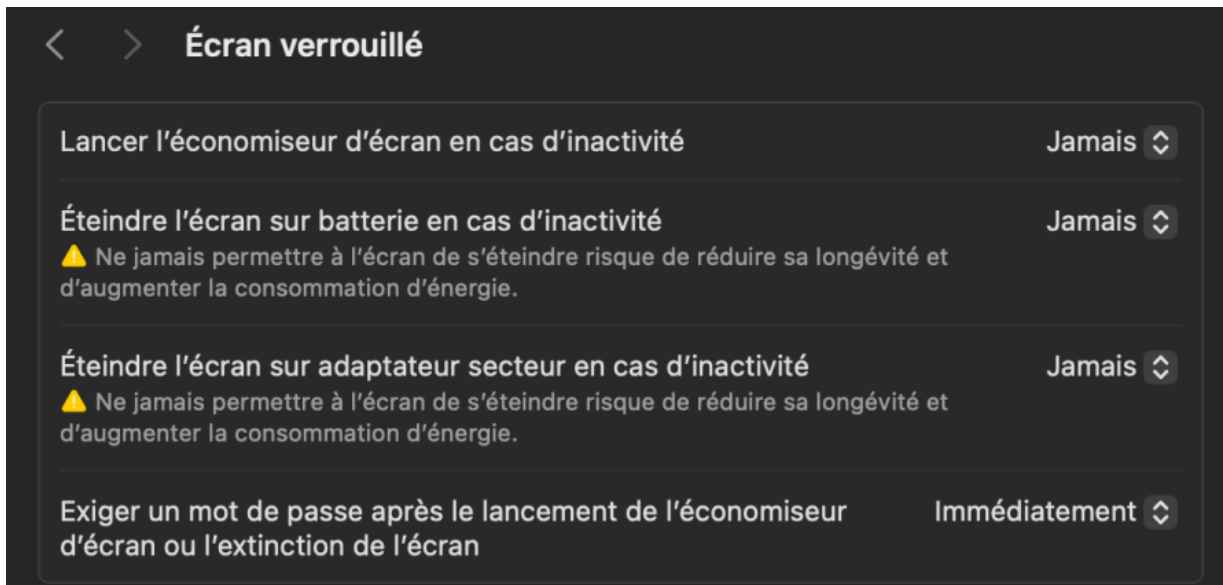
```
(.venv) demo@mac git checkout -b dev  
Basculement sur la nouvelle branche 'dev'
```

```
1 git checkout -b dev
```

⚠ Attention

Pendant toute la durée du cours, veuillez à désactiver la mise en veille automatique de votre machine. Certains téléchargements prendront 10 à 20 minutes. Le fait de désactiver la veille automatique permet d'éviter des erreurs en cours de processus.

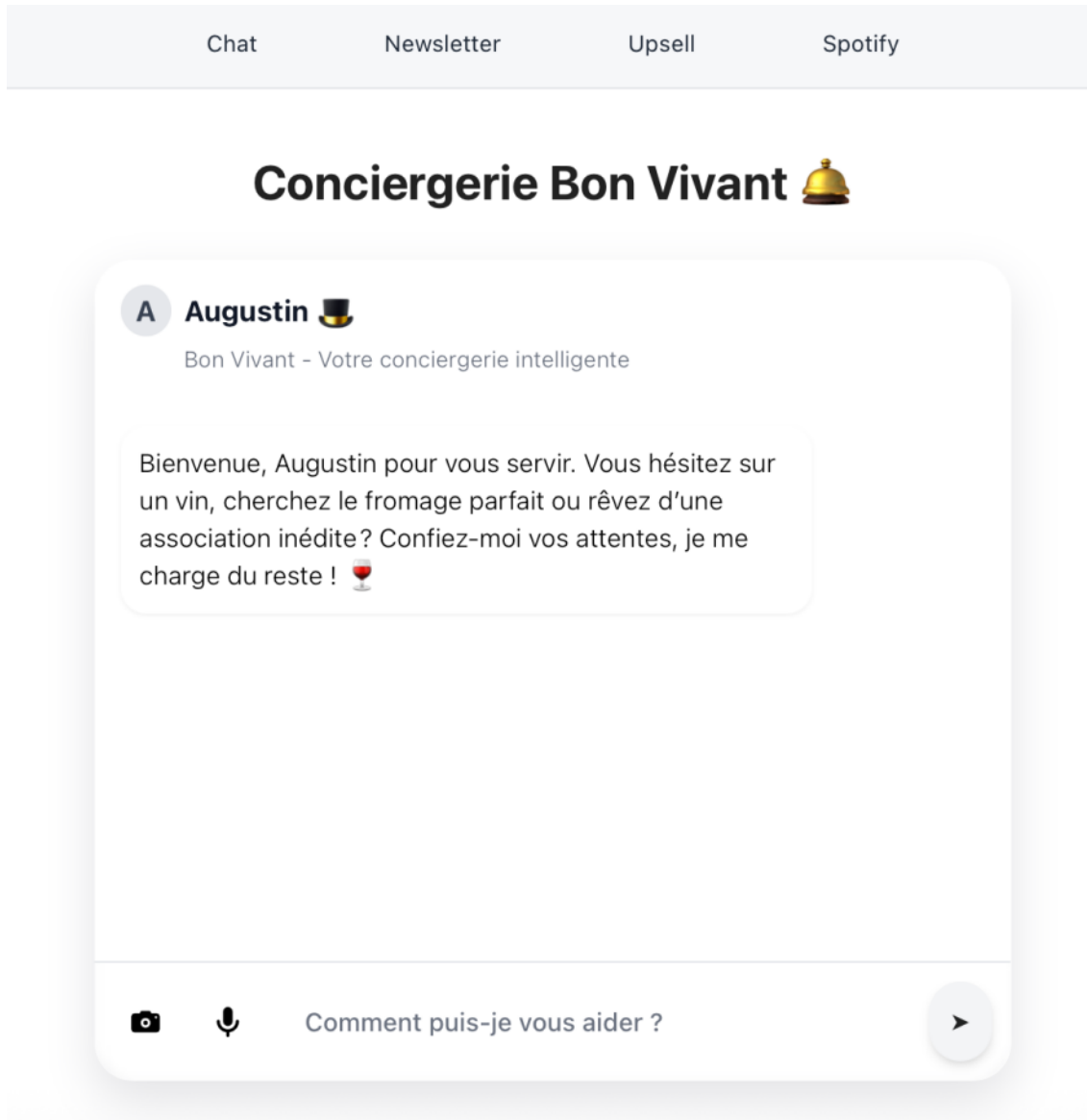
Depuis un mac, menu pomme > réglage système > écran verrouillé > Désactiver toutes les options.



Dorénavant, nous allons créer la première version de notre chatbot. Nous allons tester l'aller-retour frontend/backend avec une réponse fixe.

Cette première version évoluera tout au long du cours jusqu'à atteindre le résultat suivant, à savoir un chatbot hébergé en local capable de répondre aux requêtes d'un utilisateur de deux manières. La première, si l'utilisateur pose une question sur un produit présent dans la base de données, le chatbot répondra en adéquation avec la demande (grâce au fichier JSON).

Si l'utilisateur pose une question un produit qui n'est pas présent dans la base de données, l'IA répondra en se basant sur ses connaissances.



Commençons par créer une API minimaliste afin de comprendre la structure propre à Fast API et valider que tout fonctionne correctement.

Rendez-vous dans votre fichier main.py.

```
(.env) demo@mac cd backend
(.env) demo@mac cd api
(.env) demo@mac cd main.py
```

```
1 cd backend
2 cd api
3 cd main.py
```

Comme pour toute API, tout en haut du fichier, nous commençons par les imports nécessaires.

```
1 from fastapi import FastAPI
2 from pydantic import BaseModel
3 from typing import List, Optional
```

```
1 from fastapi import FastAPI
2 from pydantic import BaseModel
3 from typing import List, Optional
```

On crée ensuite notre application Fast API.

```
5 app = FastAPI()
```

```
1 app = FastAPI()
```

Nous poursuivons avec la création de nos classes Pydantic en renseignant ce qu'on reçoit du frontend, et ce qu'on renvoie en réponse.

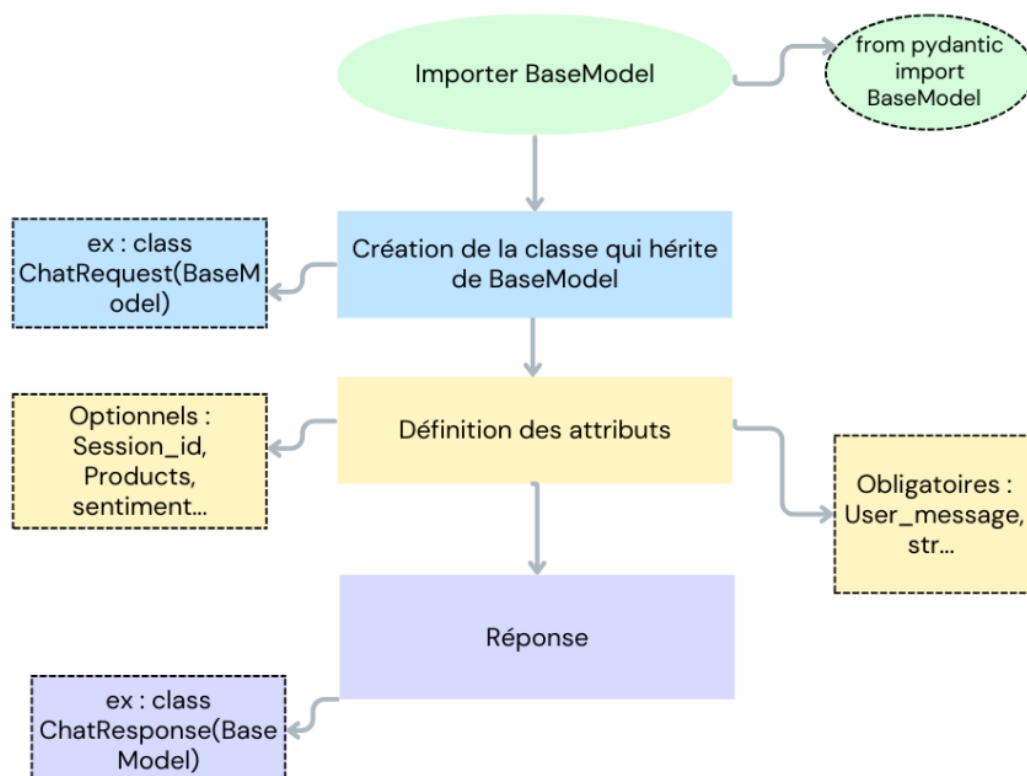
```
7 class ChatRequest(BaseModel):
8     session_id: Optional[str] = None
9     user_message: str
10    products: Optional[List[str]] = []
11
12    class ChatResponse(BaseModel):
13        recommendation: str
14        sentiment: Optional[str] = None
15        escalate_to_human: bool = False
```

Rappel

Pydantic est une librairie Python qui permet de valider et structurer les données. Elle propose différents types de modèles mais celui que nous allons utiliser ici est BaseModel.

Un BaseModel permet de définir un schéma qui décrit à quoi doivent ressembler les données échangées entre le frontend et le backend.

Cela évite d'avoir des erreurs lorsqu'un champ manque, ou encore qu'il soit du mauvais type.



Comment se structure un schéma Pydantic ?

1. Nous commençons par importer BaseModel
2. Ensuite, nous créons une classe qui hérite de BaseModel
3. Nous définissons des attributs typés (str, int, List[str], Optional[...]... voir liste complète dans la doc officielle : Pydantic⁵)
4. Nous pouvons également renseigner une valeur par défaut.

Déconstruction de la classe ChatRequest :

Construire un schéma Pydantic repose sur un ensemble d'éléments optionnels et obligatoires.

- Session_id : permet d'indiquer un identifiant de session (optionnel)
- User_message : ici on passe ce que l'utilisateur écrit au chatbot (obligatoire)
- Products : correspond à la liste de produits (optionnel et par défaut la liste est vide)

Passons maintenant à ChatResponse.

- Recommendation : il s'agit de la réponse générée par l'IA (obligatoire)
- Sentiment : il s'agit de l'analyse de sentiment (optionnel)
- Escalate_to_human : c'est un booléen qui permet de dire si le bot doit passer la main à un humain (optionnel et par défaut : false)

Ainsi, l'on voit que l'avantage de combiner Pydantic et Fast API est de pouvoir valider automatiquement les données reçues selon notre modèle Pydantic.

Si la requête ne correspond pas au schéma attendu, une erreur claire est renvoyée sans que l'on ait besoin d'écrire nous-mêmes du code de validation.

Pour aller plus loin, retrouvez la liste des autres types de modèles Pydantic à l'adresse suivante : Documentation pydantic⁶

```
1 class ChatRequest(BaseModel):  
2     session_id: Optional[str] = None  
3     user_message: str  
4     products: Optional[List[str]] = []  
5  
6 class ChatResponse(BaseModel):  
7     recommendation: str  
8     sentiment: Optional[str] = None  
9     escalate_to_human: bool = False
```

⁵. <https://docs.pydantic.dev/1.10/usage/types/>

⁶. <https://pydantic.com.cn/fr/concepts/models/#modeles-imbriques>

Enfin, nous allons créer la "route" de notre API, c'est-à-dire l'adresse que le frontend viendra interroger pour obtenir une recommandation.

Nous finissons par créer notre route ou endpoint.

```
17 @app.post("/chat", response_model=ChatResponse)
18 async def chat_endpoint(request: ChatRequest):
19     # Pour l'instant, une réponse fixe
20     return ChatResponse(
21         recommendation="Je te recommande un Chardonnay.",
22         sentiment="neutral",
23         escalate_to_human=False
24     )
25
```

```
1 @app.post("/chat", response_model=ChatResponse)
2 async def chat_endpoint(request: ChatRequest):
3     # Pour l'instant, une réponse fixe
4     return ChatResponse(
5         recommendation="Je te recommande un Chardonnay.",
6         sentiment="neutral",
7         escalate_to_human=False
8     )
```

Comment créer un endpoint ?

[Méthode](#)

On commence par définir un endpoint. Pour ce faire, nous allons utiliser un décorateur (@app.get, @app.post etc). Ceci dans le but d'indiquer l'URL et le type de requête HTTP (GET, POST, PUT, DELETE etc).

Dans notre exemple : @app.post("/chat")

```
@app.post("/chat",
```

Lorsqu'on écrit cet endpoint POST/chat, on crée une route accessible en POST à l'adresse http://localhost:8000/chat

Ensuite, nous devons créer une fonction Python.

Cette fonction sera exécutée quand un client appelle la route.

Celle-ci peut être asynchrone (async def) afin de mieux gérer plusieurs utilisateurs.

Elle peut recevoir un argument des données validées par un schéma Pydantic.

Dans notre exemple, async def chat_endpoint(request: ChatRequest) :

```
async def chat_endpoint(request: ChatRequest):
```

Ensuite, nous devons définir notre modèle Pydantic comme vu précédemment afin de valider les données en entrée (request) et structurer la réponse.

Dans notre exemple : response_model=ChatResponse.

```
@app.post("/chat", response_model=ChatResponse)
```

Cela signifie que FastAPI s'attend à recevoir en retour un objet conforme au schéma ChatResponse.

Et enfin, nous allons retourner la réponse (dictionnaire Python ou modèle Pydantic).

La fonction doit retourner un objet et FastAPI le convertira automatiquement en JSON.

Dans notre cas, on retourne une instance de classe héritée de BaseModel qui sera automatiquement transformée en réponse JSON envoyée au client.

```
@app.post("/chat", response_model=ChatResponse)
async def chat_endpoint(request: ChatRequest):
    # Pour l'instant, une réponse fixe
    return ChatResponse(
        recommendation="Je te recommande un Chardonnay.",
        sentiment="neutral",
        escalate_to_human=False
    )
```

```
1 return ChatResponse( recommendation="Je te recommande un Chardonnay.",
    sentiment="neutral", escalate_to_human=False )
```

Pour le moment, la recommandation du chatbot est fixe. (voir ligne "Je te recommande un Chardonnay.", sentiment="neutral")

Nous améliorerons cette fonction dans un second temps.

Voici le résultat final de notre chatbot sans IA :

```
1  ✓ from fastapi import FastAPI
2  from pydantic import BaseModel
3  from typing import List, Optional
4
5  app = FastAPI()
6
7  ✓ class ChatRequest(BaseModel):
8      session_id: Optional[str] = None
9      user_message: str
10     products: Optional[List[str]] = []
11
12  ✓ class ChatResponse(BaseModel):
13      recommendation: str
14      sentiment: Optional[str] = None
15      escalate_to_human: bool = False
16
17  @app.post("/chat", response_model=ChatResponse)
18  ✓ async def chat_endpoint(request: ChatRequest):
19      # Pour l'instant, une réponse fixe
20  ✓  return ChatResponse(
21      recommendation="Je te recommande un Chardonnay.",
22      sentiment="neutral",
23      escalate_to_human=False
24  )
25
```

```

1 from fastapi import FastAPI
2 from pydantic import BaseModel
3 from typing import List, Optional
4 app = FastAPI()
5 class ChatRequest(BaseModel):
6     session_id: Optional[str] = None
7     user_message: str
8     products: Optional[List[str]] = []
9
10 class ChatResponse(BaseModel):
11     recommendation: str
12     sentiment: Optional[str] = None
13     escalate_to_human: Optional[bool] = False
14
15 @app.post("/chat", response_model=ChatResponse)
16 async def chat_endpoint(request: ChatRequest):
17     # Pour l'instant, une réponse fixe
18     return ChatResponse(
19         recommendation="Je te recommande un Chardonnay.",
20         sentiment="neutral",
21         escalate_to_human=False
22 )

```

Afin de tester le bon fonctionnement du code, vous allez lancer pour la première fois votre serveur coté backend. Pour ce faire, rien de plus simple. A la racine du backend, et en ayant le venv activé il suffit de taper la commande suivante :

```

(.venv) demo@mac uvicorn api.main:app --reload
INFO:      Will watch for changes in these directories: ['/Users/beverllydorval/D
ocuments/tuto-bonvivant-project-demo/backend']
INFO:      Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
INFO:      Started reloader process [66270] using StatReload
INFO:      Started server process [66301]
INFO:      Waiting for application startup.
INFO:      Application startup complete.

```

```

1 cd backend
2 source .venv/bin/activate
3 uvicorn api.main:app --reload

```

À cette étape, vous devriez avoir dans votre terminal une ligne « *Uvicorn running on 127.0.0.1:8000⁷* ».

```

INFO:      Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
INFO:      Started reloader process [4416] using StatReload
INFO:      Started server process [4421]
INFO:      Waiting for application startup.
INFO:      Application startup complete.

```

Commandes de base du serveur Uvicorn



Les commandes de base pour le serveur Uvicorn sont "CTRL+C" pour arrêter le processus en cours. Et "uvicorn api.main:app --reload" pour relancer le terminal. Gardez ces commandes auprès de vous car nous les utiliserons tout au long du cours.

⁷ <http://127.0.0.1:8000/>

Pour tester que tout se déroule comme prévu, nous allons faire un premier test.

Après avoir démarré votre serveur Uvicorn dans votre terminal, ouvrez un second terminal.

Nous allons maintenant simuler une requête envoyée à notre endpoint /chat, exactement comme le ferait un frontend pleinement fonctionnel. Cela nous permettra de vérifier que notre API retourne bien la réponse attendue. Dans ce cas précis, la recommandation de vin « *Je te recommande un Chardonnay* » étant donné qu'il s'agit de qu'on a défini dans notre endpoint.

Pour ce faire, nous allons utiliser la commande curl.

Az Définition

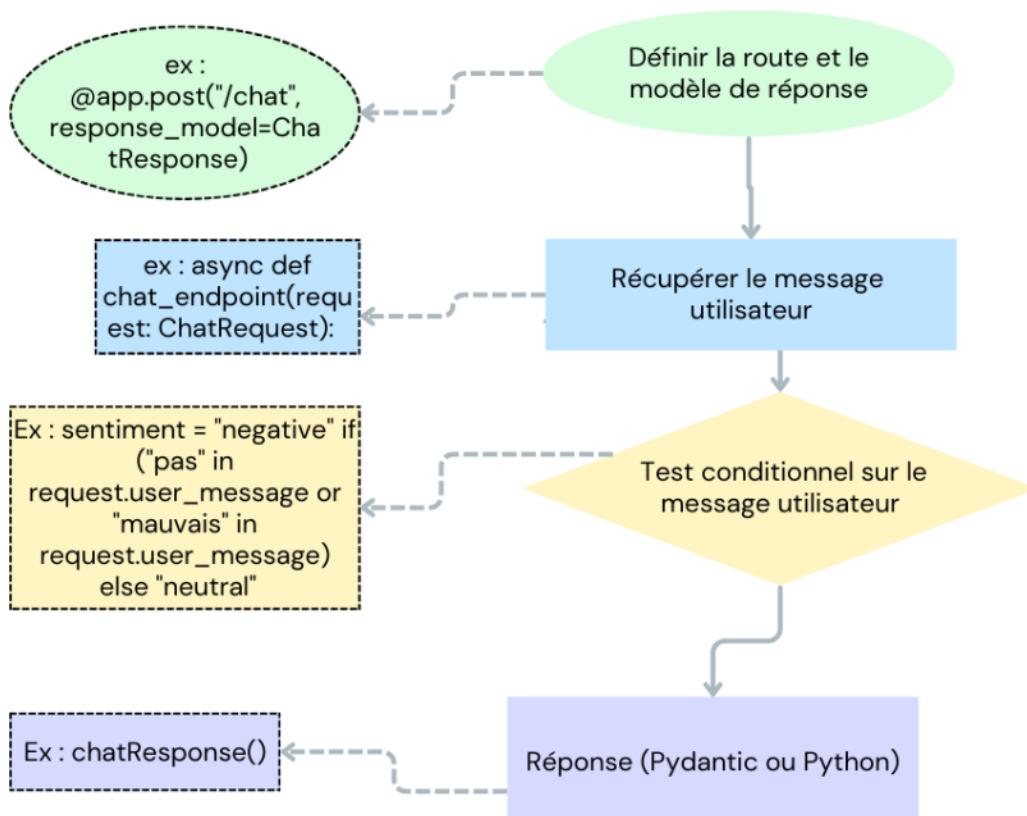
Curl est un outil en ligne de commande qui permet d'envoyer des requêtes HTTP et de visualiser directement la réponse du serveur. C'est extrêmement pratique lorsqu'on souhaite tester rapidement une API et que le frontend n'est pas encore en place.

Pour ce faire, ouvrez un second terminal et tapez la commande suivante

```
1 curl -X POST "http://127.0.0.1:8000/chat" -H "Content-Type: application/json" -d '{"user_message":"Quel vin pour le fromage ?", "products":["fromage"]}'
```

Voici le résultat attendu.

```
{"recommendation":"Je te recommande un Chardonnay.","sentiment":"neutral","escalate_to_human":false}
```



Maintenant, allons un cran plus loin. Ici, nous voulons que le chatbot fasse la différence entre une phrase neutre et une phrase négative.

Plutôt que de répondre toujours la même chose, il va essayer de détecter si l'utilisateur exprime une insatisfaction ou un rejet (par exemple s'il utilise les mots "pas" ou "mauvais"). C'est très basique, mais c'est déjà une logique plus intelligente que précédemment.

Ce qu'on vient d'ajouter c'est une condition qui analyse la question posée par l'utilisateur dans le but d'y chercher un mot "négatif".

Si c'est le cas, le champ "sentiment" passe à "negative", et "escalate_to_human" passe à True. En production, cela pourrait alerter un conseiller humain.

Ajouter une fonction d'analyse de sentiment simple

 Méthode

1. Après avoir défini la route et le modèle de réponse

```
@app.post("/chat", response_model=ChatResponse)
```

2. Nous allons récupérer le message utilisateur et ajouter un test conditionnel sur le message utilisateur. Si le message contient certains mots-clés comme "pas" ou "mauvais" on considère que le sentiment est négatif. Sinon, on considère que le sentiment est neutre.

```
async def chat_endpoint(request: ChatRequest):
    # On analyse rapidement le "sentiment" du message utilisateur
    sentiment = "negative" if ("pas" in request.user_message or "mauvais" in request.user_message) else "neutral"
```

3. On retourne ensuite un objet ChatResponse qui contient cette fois-ci notre recommandation fixe, mais aussi le résultat de notre mini analyse. Quant à lui, Escalate_to_human est un booléen qui vaut True si le sentiment est négatif afin de simuler le passage de main à un humain.

```
# On retourne la réponse (toujours fixe pour l'instant, mais enrichie du sentiment)
return ChatResponse(
    recommendation="Je te recommande un Chardonnay.",
    sentiment=sentiment,
    escalate_to_human=sentiment == "negative"
)
```

```
1 @app.post("/chat", response_model=ChatResponse)
2 async def chat_endpoint(request: ChatRequest):
3     # On analyse rapidement le "sentiment" du message utilisateur
4     sentiment = "negative" if ("pas" in request.user_message or "mauvais" in
5       request.user_message) else "neutral"
6     # On retourne la réponse (toujours fixe pour l'instant, mais enrichie du
7       sentiment)
8     return ChatResponse(
9       recommendation="Je te recommande un Chardonnay.",
10      sentiment=sentiment,
11      escalate_to_human=sentiment == "negative" )
```

Voici le résultat final :

```
from fastapi import FastAPI
from pydantic import BaseModel
from typing import List, Optional

app = FastAPI()

class ChatRequest(BaseModel):
    session_id: Optional[str] = None
    user_message: str
    products: Optional[List[str]] = []

class ChatResponse(BaseModel):
    recommendation: str
    sentiment: Optional[str] = None
    escalate_to_human: bool = False

@app.post("/chat", response_model=ChatResponse)
async def chat_endpoint(request: ChatRequest):
    # On analyse rapidement le "sentiment" du message utilisateur
    sentiment = "negative" if ("pas" in request.user_message or "mauvais" in request.user_message) else "neutral"

    # On retourne la réponse (toujours fixe pour l'instant, mais enrichie du sentiment)
    return ChatResponse(
        recommendation="Je te recommande un Chardonnay.",
        sentiment=sentiment,
        escalate_to_human=sentiment == "negative"
    )
```

Dorénavant, nous pouvons tester notre nouvelle fonction.

Commencez par relancer le serveur en arrêtant le processus en cours, puis en le relançant.

```
(.venv) demo@mac uvicorn api.main:app --reload
```

Ctrl + C

```
1 uvicorn api.main:app --reload
```

Comme précédemment, nous allons simuler une requête au backend via la commande Curl.

```
demo@mac curl -X POST "http://127.0.0.1:8000/chat" -H "Content-Type: application/json" -d '{"user_message": "Je n'aime pas le vin rouge.", "products": []}'
```

```
1 curl -X POST "http://127.0.0.1:8000/chat" -H "Content-Type: application/json" -d '{"user_message": "Je n'aime pas le vin rouge.", "products": []}'
```

Le résultat attendu est le suivant. Grâce à ces légères modifications, nous permettons au bot de déterminer le ton d'une question (voir "escalate_to_human":true).

```
e: application/json" -d '{"user_message": "Je n'aime pas le vin rouge.", "products": []}'
{"recommendation": "Je te recommande un Chardonnay.", "sentiment": "negative", "escalate_to_human": true}
```

C. Chatbot sans IA v.1.2

Construction d'un set de données

Comme vu précédemment, le cycle de vie d'un projet IA commence toujours par de la collecte de données car sans données, aucune IA ne peut apprendre, s'adapter ou fournir de réponses intéressantes. Dans ce contexte, nous allons créer un fichier JSON pour la première version de notre chatbot. Ce fichier JSON va nous permettre d'interroger la base de données via le chat et viendra remplacer la réponse codée en dur.

Comment créer un fichier JSON pour Bon Vivant 16 ?

[Méthode](#)

```
[  
{  
  "trigger": "fromage",  
  "recommendation": "Un vin blanc sec comme un Sancerre."  
},  
{  
  "trigger": "steak",  
  "recommendation": "Un Bordeaux puissant et tannique."  
}  
]
```

1. Nous commençons par ouvrir des crochets. Cela indique que nous avons une liste ou un tableau contenant plusieurs éléments. Ici, chaque élément correspond à une règle que le chatbot pourra utiliser.
2. Chaque élément de la liste est écrit entre accolades. Cela correspond à un objet JSON c'est-à-dire une donnée organisée en paires clé/valeur.

Dans notre exemple avec Bon Vivant, nous avons deux objets :

"trigger" est la clé : c'est le mot déclencheur que l'utilisateur va écrire dans le chat
"fromage" est la valeur associée. Ainsi, si l'utilisateur écrit "fromage", le chatbot saura quoi répondre.

"recommendation" est la clé : c'est-à-dire la réponse que le chatbot doit donner.

"Un vin blanc sec comme un Sancerre" est la valeur associée, à savoir le texte que verra l'utilisateur à l'écran.

Créons notre fichier JSON.

Placez-vous à la racine du projet et tapez la commande suivante.

```
(.venv) demo@mac touch backend/data/products.json
```

```
1 touch backend/data/products.json
```

```
[  
  
  {  
  
    "trigger": "fromage",  
  
    "recommendation": "Un vin blanc sec comme un Sancerre."  
  
  },  
  
  {  
  
    "trigger": "steak",  
  
    "recommendation": "Un Bordeaux puissant et tannique."  
  
  }  
  
]
```

Composez votre fichier JSON comme nous l'avons vu précédemment.

Il s'agit de produits test, que nous remplacerons dans un second temps par les produits du e-commerce Bon Vivant.

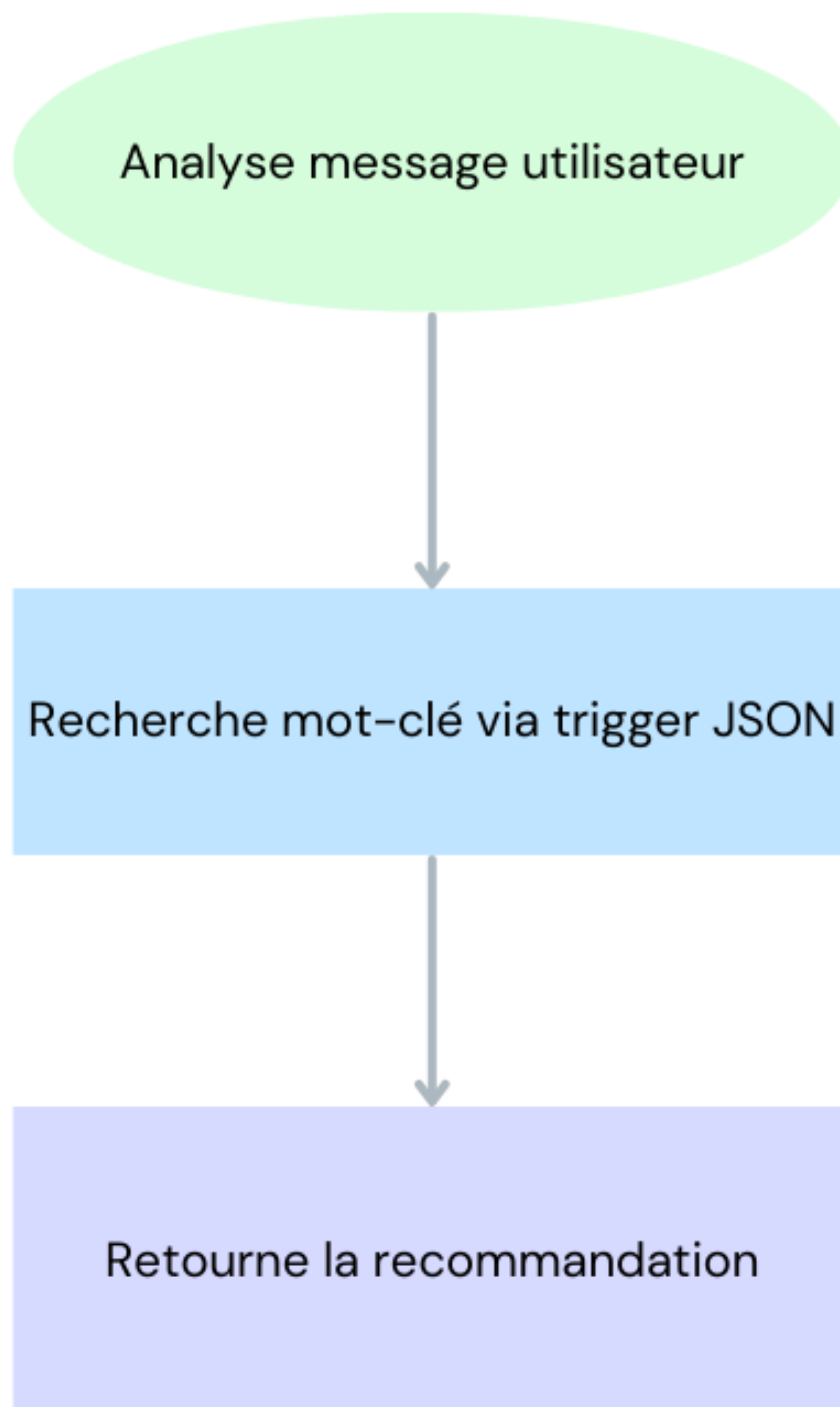
```
1 [  
2 {  
3 "trigger": "fromage",  
4 "recommendation": "Un vin blanc sec comme un Sancerre."  
5 },  
6 {  
7 "trigger": "steak",  
8 "recommendation": "Un Bordeaux puissant et tannique."  
9 }  
10 ]
```

Notre première version du chatbot est désormais terminée. Celui-ci est capable de donner une réponse fixe, ou de s'appuyer sur notre fichier JSON. C'est un bon début mais nous allons aller plus loin.

Pour ce faire, nous allons commencer par structurer un peu mieux notre projet. L'idée ici, est de préparer dès maintenant l'arrivée d'un véritable modèle d'IA sans avoir à réécrire tout le code plus tard.

Pour cela, nous allons créer un fichier `services.py`.

Ce fichier sera placé dans le dossier `core` et il contiendra la logique métier de notre chatbot.



Très concrètement, `services.py` analysera les messages utilisateurs, recherchera un mot-clé (via le trigger du JSON) et retournera la recommandation correspondante depuis notre base JSON.

Séparer la logique métier grâce au fichier `services.py`

 Méthode

1. Commencez par créer le fichier `services.py` dans `backend/core`

```
(.venv) demo@mac touch backend/core/services.py  
(.venv) demo@mac █
```

```
1 touch backend/core/services.py
```

2. Ensuite, ouvrez le fichier et importez les bibliothèques nécessaires à savoir `json` et `os`.
`json` permet de lire et manipuler des fichiers JSON en Python.

os sert à gérer les chemins de fichiers. Nous allons l'utiliser pour pointer vers notre base de données.

```
import json
import os
```

3. Dorénavant, nous allons définir le chemin qui mène à notre dataset dans une variable Python. Le but est que notre variable (DATASET_PATH) contienne le chemin exact de notre base JSON peu importe d'où l'on lance le script.

Pour cela, Python met à notre disposition une variable spéciale appelée `__file__`. Celle-ci présente le fichier Python en cours d'exécution (ici `services.py`).

Grâce à la commande `os.path.dirname(__file__)`, nous allons récupérer uniquement le dossier du fichier courant.

Grâce à `../data/products.json` nous indiquons que nous voulons remonter d'un cran vers `backend/` et aller chercher le fichier `products.json` dans le fichier `data`.

```
DATASET_PATH = os.path.join(os.path.dirname(__file__), "../data/products.json")
```

4. Ensuite, nous allons définir la fonction.

Cette fonction peut prendre en entrée un message utilisateur (`user_message`)

Elle doit retourner une recommandation si elle trouve un "trigger" dans le message, conformément à ce qui a été établi dans notre fichier JSON.

Nous finissons par ouvrir et lire notre fichier JSON

`Open(DATASET_PATH, "r")` permettra d'ouvrir le fichier en mode lecture

`Json.load(f)` permettra de lire son contenu et de le transformer en liste d'objets Python

```
def get_product_info(user_message):
    try:
        with open(DATASET_PATH, "r") as f:
            data = json.load(f)
```

5. Par la suite, nous allons rechercher un mot-clé dans le message utilisateur

Pour ce faire, on va commencer par parcourir chaque objet du JSON (`entry`)

Si le mot déclencher est trouvé dans le message utilisateur (`entry["trigger"]`), tout d'abord nous allons le transformer en minuscules avec `.lower()` afin d'éviter des erreurs de casse, et nous retournerons ensuite la recommandation associée.

```
for entry in data:
    if entry["trigger"] in user_message.lower():
        return entry["recommandation"]
```

6. La septième étape concerne la gestion des erreurs.

Si une erreur survient comme un fichier introuvable ou le JSON mal formé, le programme continue sans planter.

Dans notre cas, on ignore simplement l'erreur avec (`pass`)

7. Enfin, vient la dernière étape. Si aucun mot-clé n'est trouvé, la fonction retourne `None`. Cela permet ensuite au chatbot de répondre par défaut la réponse que nous avons codé précédemment "je te recommande un Chardonnay".

Ainsi, nous venons de créer une fonction (`get_product_info`) qui va ouvrir la base JSON, chercher si un mot déclencheur (trigger) est présent dans le message utilisateur et renvoyer la recommandation correspondante le cas échéant. Dans le cas contraire, elle renvoie `None`.

```
import json
import os

DATASET_PATH = os.path.join(os.path.dirname(__file__), "../data/products.json")

def get_product_info(user_message):
    try:
        with open(DATASET_PATH, "r") as f:
            data = json.load(f)

            for entry in data:
                if entry["trigger"] in user_message.lower():
                    return entry["recommendation"]

    except Exception:
        pass

    return None
```

```
1 import json
2 import os
3 DATASET_PATH = os.path.join(os.path.dirname(__file__), "../data/products.json")
4 def get_product_info(user_message):
5     try:
6         with open(DATASET_PATH, "r") as f:
7             data = json.load(f)
8             for entry in data:
9                 if entry["trigger"] in user_message.lower():
10                    return entry["recommendation"]
11     except Exception:
12         pass
13     return None
```

Pour terminer, nous allons remplacer la réponse fixe du chatbot par un appel à notre fonction "produit".

Si le chatbot trouve un mot-clé, il répondra en accord avec la requête. Sinon, il restera sur la recommandation par défaut.

```
@app.post("/chat", response_model=ChatResponse)
async def chat_endpoint(request: ChatRequest):
    dataset_response = get_product_info(request.user_message)

    if dataset_response:
        recommendation = dataset_response
    else:
        recommendation = "Je te recommande un Chardonnay."

    sentiment = "negative" if ("pas" in request.user_message or "mauvais" in request.user_message) else "neutral"

    return ChatResponse(
        recommendation=recommendation,
        sentiment=sentiment,
        escalate_to_human=sentiment == "negative"
    )
```

```

1 from core.services import get_product_info
2 @app.post("/chat", response_model=ChatResponse)
3 async def chat_endpoint(request: ChatRequest):
4     dataset_response = get_product_info(request.user_message)
5     if dataset_response: recommendation = dataset_response
6     else: recommendation = "Je te recommande un Chardonnay."
7     sentiment = "negative" if ("pas" in request.user_message or "mauvais" in
8         request.user_message) else "neutral"
9     return ChatResponse(
10         recommendation=recommendation,
11         sentiment=sentiment,
12         escalate_to_human=sentiment == "negative"
13     )

```

Il est maintenant temps de réaliser un autre test. Comme précédemment, relancez votre serveur (CTRL + C puis la commande Uvicorn : `uvicorn api.main:app --reload`), puis effectuez un test CURL dans un autre terminal.

```

demo@mac curl -X POST "http://127.0.0.1:8000/chat" -H "Content-Type: application/json" -d '{"user_message": "Quel vin pour le fromage ?", "products": []}'

```

```

1 curl -X POST "http://127.0.0.1:8000/chat" -H "Content-Type: application/json" -d '{"user_message": "Quel vin pour le fromage ?", "products": []}'

```

La réponse attendue est la suivante :

```

00/chat" -H "Content-Type: application/json" -d '{"user_message": "Quel vin pour le fromage ?", "products": []}'

{"recommendation": "Un vin blanc sec comme un Sancerre.", "sentiment": "neutral", "escalate_to_human": false}

```

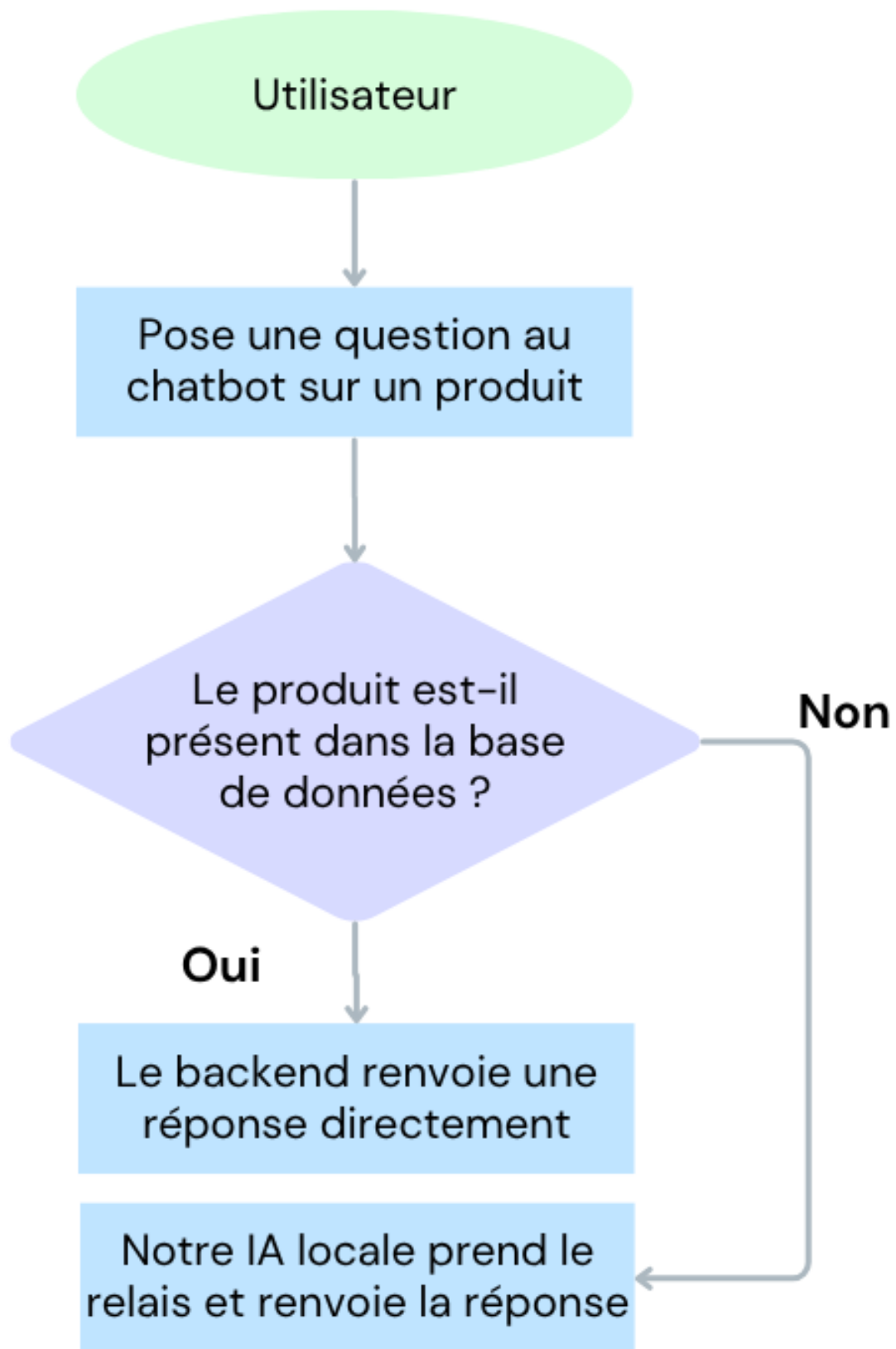
À ce stade, votre chatbot répond grâce à une base de connaissances simple.

Prochaine étape, intégrer un vrai modèle IA pour traiter les questions hors périmètre.

D. Création du chatbot (texte avec IA)

Un chatbot connecté à une base de données, c'est déjà très utile. Mais dans la vie réelle, l'utilisateur pose parfois des questions inattendues.

C'est là qu'intervient le « *fallback IA* » : lorsque la base de données ne sait pas répondre à la requête, l'IA prend le relais.



Dans le but de respecter les contraintes budgétaires avancées par l'équipe d'Émilie de Bon Vivant, nous avons fait le choix d'héberger un modèle d'IA open source en local.

Ici, Mistral-7B.

Qu'est-ce que Mistral-7B

 Rappel

Mistral 7B est un modèle de langage open-source (un LLM) créé par la startup française Mistral.

7B signifie qu'il contient 7 milliards de paramètres (cela fait référence à la mémoire interne du modèle).

C'est un modèle puissant bien que compact, capable de générer du texte en plusieurs langues dont le français.

Il est optimisé pour tourner en local.

Point intéressant, il rivalise avec des modèles plus gros sur certaines tâches comme la classification, ou encore la génération de texte. Son poids est également un avantage car il est compris en 4go et 14go. La licence est également open-source et gratuite.

Mistral a également plusieurs variantes. Il y a Mistral 7B : la version de base.

Mais aussi Mistral-7B-Instruct qui est la version optimisée pour répondre aux instructions humaines.

Q4, Q5, Q8 etc. fait référence au niveau de compression (quantized versions) c'est-à-dire, les versions compressées qui permettent de tourner plus facilement sur CPU sans trop perdre en qualité.

Q4 est le plus léger, mais un peu moins précis.

Q5 est un équilibre entre vitesse et qualité.

Q8 est plus lourd mais aussi plus fidèle au modèle d'origine.

Nous allons commencer par créer un dossier à la racine de notre backend qui contiendra notre modèle IA.

```
• demo@mac mkdir -p backend/models
```

```
• demo@mac cd backend/models
```

```
• demo@mac curl -L -o mistral-7b-q4_k_m.gguf https://huggingface.co/TheBloke/Mistral-7B-Instruct-v0.2-GGUF/resolve/main/mistral-7b-instruct-v0.2.Q4_K_M.gguf
```

```
1 mkdir -p backend/models
2 cd backend/models
```

Ensuite, depuis le dossier backend/models, nous allons installer en ligne de commande le modèle LLAMA.

```
1 curl -L -o mistral-7b-q4_k_m.gguf https://huggingface.co/TheBloke/Mistral-7B-Instruct-v0.2-GGUF/resolve/main/mistral-7b-instruct-v0.2.Q4_K_M.gguf
```

Bon à savoir

 Remarque

En fonction de votre connexion internet, le modèle peut potentiellement mettre plusieurs dizaines de minutes à se télécharger. Ce modèle est disponible gratuitement à l'adresse suivante :

Huggingface⁸

⁸ <https://huggingface.co/TheBloke/Mistral-7B-Instruct-v0.2-GGUF>

Maintenant que le modèle IA a bien été téléchargé (vérifiez dans votre dossier backend/models pour vous en assurer) il est temps de créer un wrapper.

Qu'est-ce qu'un Wrapper ?

Az Définition

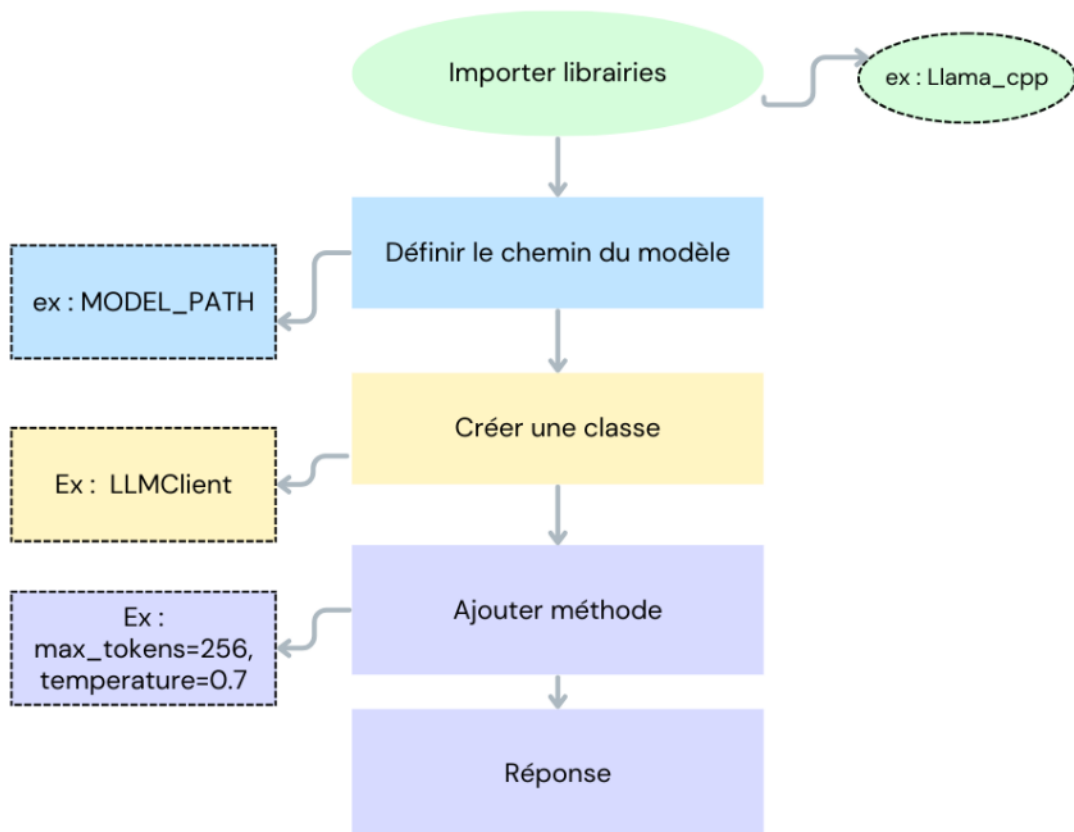
Un wrapper est une classe Python qui nous permettra d'utiliser facilement le modèle d'IA sans dupliquer le code dans tout le projet.

Pour ce faire, nous allons créer le fichier llm_client.py dans le dossier backend/core/

```
demo@mac cd core
demo@mac touch llm_client.py
```

Incorporer l'IA dans notre chatbot

Méthode



1. On commence par importer nos librairies

Llama_cpp : est une librairie Python qui permet d'interagir avec les modèles IA locaux au format GGUF tels que Mistral ou LLama.

```
from llama_cpp import Llama
import os
```

2. Deuxièmement, nous allons construire un chemin absolu vers notre modèle IA

```

BASE_DIR = os.path.dirname(os.path.abspath(__file__))
DEFAULT_MODEL_PATH = os.path.abspath(os.path.join(BASE_DIR, "../models/mistral-7b-q4_k_m.gguf"))

MODEL_PATH = os.environ.get("LLAMA_MODEL_PATH", DEFAULT_MODEL_PATH)

```

3. Ensuite, nous allons créer une classe LLMClient.

Cette classe encapsule un client LLama local.

Lorsqu'on crée un objet LLMClient(), il charge automatiquement le modèle.

Ici, les paramètres sont importants.

N_ctx=2048 : l'IA peut traiter 2048 tokens en contexte (ce qui équivaut à quelques pages de texte)

N_threads=4 : c'est le nombre de coeurs processeur utilisés (plus le chiffre est élevé, plus l'IA sera rapide par contre, cela peut ralentir le PC donc souvent, il faut tester pour trouver le bon équilibre entre vitesse et stabilité. Ici, nous avons déterminé 4 coeurs.

Verbose=false : cela évite de spammer le terminal avec des logs

```

class LLMClient:
    def __init__(self, model_path=MODEL_PATH):
        self.llm = Llama(
            model_path=model_path,
            n_ctx=2048, # longueur du contexte (tokens pris en compte par l'IA)
            n_threads=4, # à adapter selon la puissance de votre machine
            verbose=False
        )

```

3. Ensuite, nous allons ajouter une méthode afin de compléter le texte.

Prompt: cela correspond au texte que l'on va donner au modèle

Maxtokens=256 : cela limite la taille de la réponse générée (ici 256 tokens maximum)

Temperature=0.7 permet de contrôler la créativité. Plus le score est bas plus il est rigide.

Stop=["</s>"] indique un token d'arrêt. Ici la balise en fin de phrase.

Enfin la méthode retourne uniquement le texte généré et nettoyé (.strip())

```

def complete(self, prompt, max_tokens=256, temperature=0.7):
    output = self.llm(
        prompt,
        max_tokens=max_tokens,
        temperature=temperature,
        stop=["</s>"]
    )
    return output['choices'][0]['text'].strip()

```

```

1 from llama_cpp import Llama
2 import os
3 BASE_DIR = os.path.dirname(os.path.abspath(__file__))
4 DEFAULT_MODEL_PATH = os.path.abspath(os.path.join(BASE_DIR, "../models/mistral-7b-q4_k_m.gguf"))
5 MODEL_PATH = os.environ.get("LLAMA_MODEL_PATH", DEFAULT_MODEL_PATH)
6 class LLMClient:
7     def __init__(self, model_path=MODEL_PATH):
8         self.llm = Llama(
9             model_path=model_path, n_ctx=2048, # longueur du contexte (tokens pris en compte par l'IA)
10             n_threads=4, # à adapter selon la puissance de votre machine
11             verbose=False
12         )
13     def complete(self, prompt, max_tokens=256, temperature=0.7):
14         output = self.llm(
15             prompt,
16             max_tokens=max_tokens,
17             temperature=temperature,
18             stop=["</s>"]
19         )
20         return output['choices'][0]['text'].strip()

```



```

10         max_tokens=max_tokens,
11         temperature=temperature,
12         stop=["</s>"]
13     )
14     return output['choices'][0]['text'].strip()

```

```

from llama_cpp import Llama
import os

BASE_DIR = os.path.dirname(os.path.abspath(__file__))
DEFAULT_MODEL_PATH = os.path.abspath(os.path.join(BASE_DIR, "../models/mistral-7b-q4_k_m.gguf"))

MODEL_PATH = os.environ.get("LLAMA_MODEL_PATH", DEFAULT_MODEL_PATH)

class LLMClient:
    def __init__(self, model_path=MODEL_PATH):
        self.llm = Llama(
            model_path=model_path,
            n_ctx=2048,      # longueur du contexte (tokens pris en compte par l'IA)
            n_threads=4,     # à adapter selon la puissance de votre machine
            verbose=False
        )

    def complete(self, prompt, max_tokens=256, temperature=0.7):
        output = self.llm(
            prompt,
            max_tokens=max_tokens,
            temperature=temperature,
            stop=["</s>"]
        )
        return output['choices'][0]['text'].strip()

```

Par la suite, nous allons modifier notre endpoint /chat présent dans le fichier main.py afin qu'il fasse appel au wrapper quand la base de données ne trouve pas la requête de l'utilisateur.

Nous conservons notre logique précédente qui consiste à rechercher une réponse dans la base de données locale, et dans le cas contraire, nous demandons à l'IA locale de répondre à la requête.

Nous commençons par regarder si la base de données locale connaît la réponse. Si ce n'est pas le cas, nous construisons un prompt et demandons à l'IA de répondre.

Cet appel est asynchrone grâce à `run_in_threadpool`, afin de ne pas bloquer le serveur Fast API.

```

from core.llm_client import LLMClient
from starlette.concurrency import run_in_threadpool

llm_client = LLMClient()

@app.post("/chat", response_model=ChatResponse)
async def chat_endpoint(request: ChatRequest):
    dataset_response = get_product_info(request.user_message)

    if dataset_response:
        recommendation = dataset_response
    else:
        prompt = f"L'utilisateur demande : {request.user_message}. Que recommandes-tu comme vin ?"
        recommendation = await run_in_threadpool(llm_client.complete, prompt)

    sentiment = "negative" if ("pas" in request.user_message or "mauvais" in request.user_message) else "neutral"

    return ChatResponse(
        recommendation=recommendation,
        sentiment=sentiment,
        escalate_to_human=sentiment == "negative"
    )

```

```

1 from core.llm_client import LLMClient from starlette.concurrency import
   run_in_threadpool
2 llm_client = LLMClient()
3 @app.post("/chat", response_model=ChatResponse)
4 async def chat_endpoint(request: ChatRequest):
5     dataset_response = get_product_info(request.user_message)
6     if dataset_response:
7         recommendation = dataset_response
8     else:
9         prompt = f"L'utilisateur demande : {request.user_message}. Que recommandes-tu comme
   vin ?" recommendation = await run_in_threadpool(llm_client.complete, prompt)
10 sentiment = "negative" if ("pas" in request.user_message or "mauvais" in
   request.user_message) else "neutral"
11
12 return ChatResponse(
13     recommendation=recommendation,
14     sentiment=sentiment,
15     escalate_to_human=sentiment == "negative"
16 )

```

Il est temps de tester la fonction !

Comme précédemment, arrêtez puis relancez le serveur Uvicorn.

Ensuite, nous allons effectuer 2 tests curl qui permettront de nous assurer que dans un premier cas la fonction fait appel au dataset et dans un autre que c'est l'IA qui répond.

Commençons par le premier cas. Pour s'assurer que la requête soit liée au contenu du dataset, il suffit d'effectuer une requête relative à un produit de la base de données.

```

demo@mac curl -X POST "http://127.0.0.1:8000/chat" \
-H "Content-Type: application/json" \
-d '{"user_message": "Quel vin pour le fromage ?",
  "products": []}'

```

```

1 curl -X POST "http://127.0.0.1:8000/chat" \
2 -H "Content-Type: application/json" \
3 -d '{
4     "user_message": "Quel vin pour le fromage ?",
5     "products": []
6 }'

```

Ici le résultat attendu est "recommendation": "Un vin blanc sec comme un Sancerre." Car il s'agit du contenu de la base de données.

```

00/chat" \
-H "Content-Type: application/json" \
-d '{"user_message": "Quel vin pour le fromage ?", "products": []}'

{"recommendation": "Un vin blanc sec comme un Sancerre.", "sentiment": "neutral", "escalate_to_human": false}

```

Ensuite, passons au second test.

Pour s'assurer que la réponse soit générée par l'IA nous allons taper la commande suivante. Le temps de réponse devrait être plus long que la précédente requête (parfois plusieurs dizaines de minutes) car le modèle IA charge dans votre projet. Les prochaines requêtes seront plus rapides.

```

demo@mac curl -X POST "http://127.0.0.1:8000/chat" \
-H "Content-Type: application/json" \
-d '{"user_message": "Quel vin pour un apéro d'été",
  "products": []}'

```

```

1 curl -X POST "http://127.0.0.1:8000/chat" \
2 -H "Content-Type: application/json" \
3 -d '{
4     "user_message": "Quel vin pour un apéro d'été",
5     "products": []
6 }'

```

```
-H "Content-Type: application/json" \
-d '{"user_message":"Quel vin pour un apéro dinatoire ?", "products":[]}'

{"recommendation":"Je suis heureux de vous aider avec votre question! Pour un apéritif dinatoire, je recommanderais un vin blanc sec et léger, idéalement avec une acidité fraîche pour stimuler l'appétit. Voici quelques options de vins blancs qui pourraient vous convaincre :\n\n1. Sauvignon Blanc : C'est un vin blanc léger et aromatique produit principalement en France dans la région de Loire, ainsi qu'en Nouvelle-Zélande. Il a souvent une acidité élevée, ce qui le rend idéal pour un apéritif.\n2. Pinot Grigio/Pinot Gris : Ce vin blanc est produit dans de nombreuses régions du monde, notamment en Italie et en Allemagne. Il est typiquement sec, léger et aromatique, avec des arômes de fruits comme la poire, la pêche ou la mangue.\n3. Chardonnay : C'est un vin blanc populaire produit dans de nombreuses régions du monde. Il peut être produit dans des", "sentiment":"neutral", "escalate_to_human":false}
```

Étant donné que l'apéro dinatoire n'est pas dans la base de données, la réponse doit être suggérée par l'IA.

II Refactorisation du backend

A. Comment passer d'une démo à un modèle scalable ?

Pour passer d'une démo à un modèle scalable, nous allons tout d'abord commencer par refactoriser la structure du projet.

Au début du cours, tout le code était concentré dans un seul fichier `main.py`.

Cela a l'avantage d'être plus rapide mais il y a aussi un plus grand risque de bugs, et de conflits.

Progressivement, nous avons créé d'autres dossiers.

Chaque dossier a une responsabilité qui lui est propre.

Rappel des dossiers et fichiers principaux

Az Définition

- API contient nos endpoints.
- Core contient notre logique IA.
- Data contient notre dataset.
- Models contient nos fichiers modèles.
- Prompts contient la gestion de nos prompts.
- Schemas.py rassemble nos modèles Pydantic (structure des données), et permet de garantir des échanges stables et lisibles entre front et back.

À cette structure, nous allons ajouter de nouveaux fichiers afin d'atteindre ce résultat.

```
backend/  
├── api/  
│   ├── __init__.py  
│   └── main.py  
├── core/  
│   ├── __init__.py  
│   ├── llm_client.py  
│   └── services.py  
├── data/  
│   └── products.json  
├── models/  
│   └── mistral-7b-instruct-v0.2.Q4_K_M.gguf  
└── requirements.txt
```

Commençons par ajouter un fichier `__init__.py` à la racine de notre projet pour que Python reconnaisse le dossier comme un package.

```
demo@mac touch __init__.py
```

```
1 touch __init__.py
```

Ensuite, à la racine du backend et venv activé, nous allons générer un fichier requirements.txt. Cela permettra de figer l'état actuel de nos dépendances.

```
demo@mac pip freeze > requirements.txt
```

```
1 pip freeze > requirements.txt
```

```
demo@mac cd api
```

```
demo@mac touch __init__.py
```

Ensuite, dans le dossier api nous allons créer à nouveau un fichier __init__.py

```
1 cd api
2 touch __init__.py
```

Puis nous allons ajouter un fichier schemas.py. Ce fichier hébergera tous nos modèles.

```
demo@mac touch schemas.py
```

Pydantic utilisés dans les endpoints.

```
1 touch schemas.py
```

Enfin, dans le dossier core nous allons créer un fichier prompt_manager.py afin de gérer nos différents prompts. Toujours dans le même dossier (core) nous allons créer un dossier prompts/. Celui-ci permettra de centraliser tous nos différents fichiers de prompts.

```
demo@mac cd ../
```

```
demo@mac cd core
```

```
demo@mac touch prompt_manager.py
```

```
demo@mac mkdir prompts
```

```
1 cd ../
2 cd core
3 touch prompt_manager.py
4 mkdir prompts
```

Ceci étant fait, nous pouvons dès à présent commencer à modifier la logique de notre application.

Nous allons commencer par déplacer tous les modèles Pydantic de main.py vers api/schemas.py

Votre fichier schemas.py doit être similaire à ceci, à savoir contenir les classes Pydantic que nous avons créé précédemment (ChatRequest et ChatResponse).

```
1 from pydantic import BaseModel
2 from typing import List, Optional
3
4
5 class ChatRequest(BaseModel):
6     session_id: Optional[str] = None
7     user_message: str
8     products: Optional[List[str]] = []
9
10 class ChatResponse(BaseModel):
11     recommendation: str
12     sentiment: Optional[str] = None
13     escalate_to_human: Optional[bool] = False
```

```

from pydantic import BaseModel
from typing import List, Optional

class ChatRequest(BaseModel):
    session_id: Optional[str] = None
    user_message: str
    products: Optional[List[str]] = []

class ChatResponse(BaseModel):
    recommendation: str
    sentiment: Optional[str] = None
    escalate_to_human: bool = False

```

De retour dans notre fichier main.py nous pouvons donc supprimer ces deux blocs de code et garder uniquement "from fastapi import FastAPI".

Ensuite, nous allons importer nos deux classes Pydantic.

```

from fastapi import FastAPI
from api.schemas import ChatRequest, ChatResponse

```

```

1 from fastapi import FastAPI
2 from api.schemas import ChatRequest, ChatResponse

```

Ces premières modifications réalisées, nous pouvons passer au frontend.

Pour ce faire, nous allons modifier notre fichier main.py afin que notre application accepte les requêtes effectuées via un navigateur grâce à un middleware CORS.

```

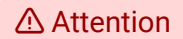
origins = [
    "http://localhost:5173",
    "http://127.0.0.1:5173"
]

```

```

app.add_middleware(
    CORSMiddleware,
    allow_origins=origins,
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)

```



Placez bien le bloc CORS en fin de votre fichier main.py

```
1 origins = #On définit une liste d'origines autorisées
2     "http://localhost:5173", "http://127.0.0.1:5173"
3 ]
4
5 app.add_middleware( # On ajoute un middleware
6     CORSMiddleware, # middleware fourni par FastAPI/Starlette afin de gérer les
7     règles CORS
8     allow_origins=origins, #seules les adresses listées dans origins sont autorisées
9     allow_credentials=True, # Permet d'envoyer des cookies ou tokens
10    d'authentification avec les requêtes
11    allow_methods=["*"], # Autorise toutes les méthodes HTTP (GET, POST, PUT...)
12    allow_headers=["*"], # Autorise tous les headers exemple : Content-Type ou
13    Authorization
14 )
```

La classe origins partage les localhost que nous allons utiliser puis le middleware CORS autorise les requêtes *via* le navigateur.

Après ces modifications, votre fichier main.py devrait être similaire a ceci.

```
1 from fastapi import FastAPI
2 from api.schemas import ChatRequest, ChatResponse
3 from core.services import get_product_info
4 from core.llm_client import LLMClient
5 from starlette.concurrency import run_in_threadpool
6 from fastapi.middleware.cors import CORSMiddleware
7
8
9 app = FastAPI()
10 llm_client = LLMClient()
11
12 origins = [
13     "http://localhost:5173",
14     "http://127.0.0.1:5173"
15 ]
16
17 @app.post("/chat", response_model=ChatResponse)
18 async def chat_endpoint(request: ChatRequest):
19     dataset_response = get_product_info(request.user_message)
20
21     if dataset_response:
22         recommendation = dataset_response
23     else:
24         prompt = f"L'utilisateur demande : {request.user_message}. Que recommandes-tu comme
25         vin ?"
26         recommendation = await run_in_threadpool(llm_client.complete, prompt)
27
28     sentiment = "negative" if ("pas" in request.user_message or "mauvais" in
29         request.user_message) else "neutral"
30
31     return ChatResponse(
32         recommendation=recommendation,
33         sentiment=sentiment,
34         escalate_to_human=sentiment == "negative"
35     )
36 app.add_middleware(
```

```

37 CORSMiddleware,
38 allow_origins=origins,
39 allow_credentials=True,
40 allow_methods=["*"],
41 allow_headers=["*"],
42 )

```

```

from fastapi import FastAPI
from api.schemas import ChatRequest, ChatResponse
from core.services import get_product_info
from core.llm_client import LLMClient
from starlette.concurrency import run_in_threadpool
from fastapi.middleware.cors import CORSMiddleware

app = FastAPI()
llm_client = LLMClient()

origins = [
    "http://localhost:5173",
    "http://127.0.0.1:5173"
]

@app.post("/chat", response_model=ChatResponse)
async def chat_endpoint(request: ChatRequest):
    dataset_response = get_product_info(request.user_message)

    if dataset_response:
        recommendation = dataset_response
    else:
        prompt = f"L'utilisateur demande : {request.user_message}. Que recommandes-tu comme vin ?"
        recommendation = await run_in_threadpool(llm_client.complete, prompt)

    sentiment = "negative" if ("pas" in request.user_message or "mauvais" in request.user_message) else "neutral"

    return ChatResponse(
        recommendation=recommendation,
        sentiment=sentiment,
        escalate_to_human=sentiment == "negative"
    )

app.add_middleware(
    CORSMiddleware,
    allow_origins=origins,
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)

```

B. Comment connecter le backend au frontend ?

Émilie a maintenant un backend IA fonctionnel, capable de répondre à toutes sortes de requêtes. Mais tant que l'équipe ne peut pas utiliser le chatbot via une interface, tout reste invisible. Il est donc temps de relier l'interface utilisateur à notre backend.

L'essentiel de ce cours sera basé sur le développement du backend en priorité, donc ici, nous allons simplement vous inviter à modifier votre frontend en fonction des indications données.

La première étape consiste à installer routeur. Routeur va nous permettre de créer notre SPA (Single Page Application) et donc de séparer nos différentes fonctionnalités par des onglets. Il y aura une page pour chaque endpoint (chatbot, newsletter, upsell).

Pourquoi installer react-router-dom ?

Az Définition

Sans routeur, tout votre projet serait sur une seule page. Avec react-router-dom, vous allez pouvoir séparer chaque fonctionnalité en onglets : Chatbot, Newsletter, Upsell.

```
demo@mac cd ..
```

```
● demo@dev cd frontend
```



```
demo@dev npm install react-router-dom

added 49 packages, and audited 297 packages in 2s

61 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
```

Ainsi, l'UX sera claire et le code sera mieux organisé.

```
1 cd ..
2 cd frontend
3 npm install react-router-dom
```

Nous allons désormais créer tous les fichiers et dossiers dont nous aurons besoin.

```
demo@dev cd src
```

```
demo@dev mkdir components
```

```
demo@dev cd components
```

```
demo@dev touch Chat.jsx Icon.jsx Navbar.jsx NewsletterIA.jsx Upsell.jsx
```

```
demo@dev cd ..
```

```
demo@dev mkdir pages
```

```
demo@dev cd pages
```

```
demo@dev touch ChatPage.jsx NewsletterPage.jsx UpsellPage.jsx
```

```
demo@dev cd ..
```

```
1 cd src
2 mkdir components
3 cd components
4 touch Chat.jsx Icon.jsx Navbar.jsx NewsletterIA.jsx Upsell.jsx
5 cd ..
6 mkdir pages
7 cd pages
8 touch ChatPage.jsx NewsletterPage.jsx UpsellPage.jsx
```

Ensuite la prochaine étape va être de styliser notre application.

```
1 cd ..
```

Nous allons modifier l'ensemble de nos fichiers présents à la racine du projet. A savoir, App.jsx, index.css et main.jsx.

```
import { BrowserRouter, Routes, Route } from "react-router-dom";
import ChatPage from "../pages/ChatPage";
import Navbar from "../components/Navbar";

export default function App() {
  return (
    <BrowserRouter>
      <div className="min-h-screen bg-white">
        { /* NAVBAR importée */ }
        <Navbar />

        <main className="flex justify-center items-start py-10">
          <Routes>
            <Route path="/" element={<ChatPage />} />
          </Routes>
        </main>
      </div>
    </BrowserRouter>
  );
}
```

App.jsx

```
1 import { BrowserRouter, Routes, Route } from "react-router-dom";
2 import ChatPage from "../pages/ChatPage";
3 import Navbar from "../components/Navbar";
4
5 export default function App() {
6   return (
7     <BrowserRouter>
8       <div className="min-h-screen bg-white">
9         { /* NAVBAR importée */ }
10      <Navbar />
11
12      <main className="flex justify-center items-start py-10">
13        <Routes>
14          <Route path="/" element={<ChatPage />} />
15        </Routes>
16      </main>
17    </div>
18  </BrowserRouter>
19 );
20 }
```

```
import { StrictMode } from 'react'
import { createRoot } from 'react-dom/client'
import './index.css'
import App from './App.jsx'

createRoot(document.getElementById('root')).render(

  <StrictMode>
    <App />
  </StrictMode>,
)
```

main.jsx

```
1 import { StrictMode } from 'react'
2 import { createRoot } from 'react-dom/client'
3 import './index.css'
4 import App from './App.jsx'
5
6 createRoot(document.getElementById('root')).render(
7 <StrictMode>
8 <App />
9 </StrictMode>,
10 )
```

```
@tailwind base;
@tailwind components;
@tailwind utilities;

:root {
  font-family: system-ui, Avenir, Arial, sans-serif;
  line-height: 1.5;
  font-weight: 300;
  color: rgba(0, 0, 0, 0.87);
  font-synthesis: none;
  text-rendering: optimizeLegibility;
  -webkit-font-smoothing: antialiased;
  -moz-osx-font-smoothing: grayscale;
}

button {
  border-radius: 8px;
  border: 1px solid transparent;
  padding: 0.6em 1.2em;
  font-size: 1em;
  font-weight: 500;
  font-family: inherit;
  background-color: #1a1a1a;
  cursor: pointer;
  transition: border-color 0.25s;
}

button:focus,
button:focus-visible {
  outline: 4px auto transparent;
  outline-color: #1a1a1a;
}

@media (prefers-color-scheme: light) {
  :root {
    color: #213547;
    background-color: #ffffff;
  }

  a:hover {
    color: #747bff;
  }

  button {
    background-color: #f9f9f9;
  }
}
```

Changed lines

index.css

```

1 @tailwind base;
2 @tailwind components;
3 @tailwind utilities;
4
5
6
7 :root {
8   font-family: system-ui, Avenir, Arial, sans-serif;
9   line-height: 1.5;
10  font-weight: 300;
11  color: rgba(0, 0, 0, 0.87);
12  font-synthesis: none;
13  text-rendering: optimizeLegibility;
14  -webkit-font-smoothing: antialiased;
15  -moz-osx-font-smoothing: grayscale;
16 }
17
18 button {
19   border-radius: 8px;
20   border: 1px solid transparent;
21   padding: 0.6em 1.2em;
22   font-size: 1em;
23   font-weight: 500;
24   font-family: inherit;
25   background-color: #1a1a1a;
26   cursor: pointer;
27   transition: border-color 0.25s;
28 }
29
30 button:focus,
31 button:focus-visible {
32   outline: 4px auto -webkit-focus-ring-color;
33 }
34
35 @media (prefers-color-scheme: light) {
36   :root {
37     color: #213547;
38     background-color: #ffffff;
39   }
40   a:hover {
41     color: #747bff;
42   }
43   button {
44     background-color: #f9f9f9;
45   }
46 }

```

Maintenant que nos éléments css de base ont été établis, nous allons passer au dossier components.



```
1 cd components
```

Puis nous allons structurer le css de notre projet.

```
frontend > src > components > Chat.jsx > ...
1 import React, { useState, useEffect, useRef } from 'react';
2 import { CameraIcon, MicroIcon } from "../Icon";
3
4 // Composant principal du chat
5 export default function Chat() {
6   // --- Etats gérés par React ---
7   const [message, setMessage] = useState(''); // Message texte tapé par l'utilisateur
8   const [file, setFile] = useState(null); // Fichier image sélectionné (upload photo)
9   const [audioFile, setAudioFile] = useState(null); // Fichier audio sélectionné (upload voix)
10  const [audioUrl, setAudioUrl] = useState(''); // URL pour la lecture audio générée par le bot
11  const [chatLog, setChatLog] = useState([
12    {
13      from: 'bot',
14      text: 'Bienvenue, Augustin pour vous servir. Vous hésitez sur un vin, cherchez le fromage',
15    }
16  ]);
17  const [isLoading, setIsLoading] = useState(false); // Etat pour afficher le loader
18
19  // --- Refs pour gestion du scroll et de l'audio ---
20  const bottomRef = useRef(); // Pour scroller en bas du chat
21  const audioRef = useRef(); // Pour piloter la lecture audio générée
22
23  // Désactive le bouton si rien n'est saisi ou si envoi en cours
24  const disableSend = (!message.trim() && !file && !audioFile) || isLoading;
25
```

Chat.jsx

```
1 import React, { useState, useEffect, useRef } from 'react';
2 import { CameraIcon, MicroIcon } from "../Icon";
3
4 // Composant principal du chat
5 export default function Chat() {
6   // --- Etats gérés par React ---
7   const [message, setMessage] = useState(''); // Message texte tapé par l'utilisateur
8   const [file, setFile] = useState(null); // Fichier image sélectionné (upload photo)
9   const [audioFile, setAudioFile] = useState(null); // Fichier audio sélectionné
10  (upload voix)
11  const [audioUrl, setAudioUrl] = useState(''); // URL pour la lecture audio générée
12  par le bot
13  const [chatLog, setChatLog] = useState([ // Historique des messages (utilisateur &
14  bot)
15  { from: 'bot', text: 'Bienvenue, Augustin pour vous servir. Vous hésitez sur un vin,
16  cherchez le fromage parfait ou rêvez d'une association inédite ? Confiez-moi vos
17  attentes, je me charge du reste ! 🍷' }
18  ]);
19  const [isLoading, setIsLoading] = useState(false); // Etat pour afficher le loader
20  (animation attente)
21
22  // --- Refs pour gestion du scroll et de l'audio ---
23  const bottomRef = useRef(); // Pour scroller en bas du chat à chaque message
24  const audioRef = useRef(); // Pour piloter la lecture audio générée par le bot
25
26  // Désactive le bouton d'envoi si rien n'est saisi ou si une requête est en cours
27  const disableSend = (!message.trim() && !file && !audioFile) || isLoading;
28
29  // --- Effet : scroll en bas automatiquement à chaque nouveau message
30  useEffect(() => {
31    bottomRef.current?.scrollIntoView({ behavior: 'smooth' });
32  }, [chatLog]);
33
34  // --- Effet : nettoyage mémoire des objets audioUrl après usage
35  useEffect(() => {
36    return () => { if (audioUrl) URL.revokeObjectURL(audioUrl); }
37  }, [audioUrl]);
38
39  // --- Fonction d'envoi principal (texte, image, ou audio)
40  async function handleSend() {

```

```

35 // 1. Cas de l'envoi d'un audio (voix utilisateur)
36 if (audioFile) {
37   setChatLog(prev => [...prev, { from: 'user', text: '(Audio envoyé)', type: 'audio'
38     }]);
39   setIsLoading(true);
40   try {
41     // On prépare la requête multipart pour le backend
42     const data = new FormData();
43     data.append('audio', audioFile);
44     const res = await fetch('http://127.0.0.1:8000/voice-chat', { method: 'POST', body:
45       data });
46     const { transcript, ai_answer, audio_filename } = await res.json();
47     // Affiche la transcription texte, puis la réponse IA dans l'historique
48     setChatLog(prev => [...prev, { from: 'user', text: transcript }, { from: 'bot', text:
49       ai_answer }]);
50     // Si un audio de réponse est généré, prépare sa lecture auto
51     if (audio_filename) {
52       const blob = await (await
53         fetch(`http://127.0.0.1:8000/audio/${audio_filename}`)).blob();
54       const url = URL.createObjectURL(blob);
55       setAudioUrl(url);
56       setTimeout(() => audioRef.current?.play(), 200);
57     } catch {
58       setChatLog(prev => [...prev, { from: 'bot', text: "Oops, l'analyse audio a échoué."
59         }]);
60       setIsLoading(false);
61       setAudioFile(null); setFile(null); setMessage('');
62       return;
63     }
64     // 2. Cas de l'envoi d'une image/photo
65     if (file) {
66       // Affiche le thumbnail dans le chat dès l'envoi
67       const photoURL = URL.createObjectURL(file);
68       setChatLog(prev => [...prev, { from: 'user', text: '(Photo envoyée)', type: 'image',
69         url: photoURL }]);
70       setIsLoading(true);
71       try {
72         // Upload de l'image vers l'API backend pour analyse
73         const data = new FormData();
74         data.append('file', file);
75         const res = await fetch('http://127.0.0.1:8000/chat/photo', { method: 'POST', body:
76           data });
77         const { description } = await res.json();
78         setChatLog(prev => [...prev, { from: 'bot', text: description }]);
79       } catch {
80         setChatLog(prev => [...prev, { from: 'bot', text: "Oops, l'analyse d'image a échoué."
81           }]);
82       }
83       setIsLoading(false);
84       setFile(null); setAudioFile(null); setMessage('');
85       return;
86     }
87     // 3. Cas classique : envoi d'un message texte
88     if (message.trim()) {
89       setChatLog(prev => [...prev, { from: 'user', text: message }]);
90       setMessage('');
91       setIsLoading(true);
92       try {
93         // Envoi vers l'endpoint principal du chatbot texte

```

```

92 const res = await fetch('http://127.0.0.1:8000/chat', {
93   method: 'POST',
94   headers: { 'Content-Type': 'application/json' },
95   body: JSON.stringify({ user_message: message }),
96 });
97 const { recommendation } = await res.json();
98 setChatLog(prev => [...prev, { from: 'bot', text: recommendation }]);
99 } catch {
100 setChatLog(prev => [...prev, { from: 'bot', text: "Oops, le serveur est injoignable."
    }]);
101 }
102 setIsLoading(false);
103 }
104 setFile(null); setAudioFile(null);
105 }
106
107 // --- Rendu UI du chat ---
108 return (
109 <div className="flex flex-col max-w-xl h-[70vh] mx-auto rounded-3xl shadow-
    [0_12px_30px_0_rgba(40,40,60,0.13)] bg-gray-150 overflow-hidden">
110
111   {/* Header de la conciergerie */}
112   <div className="backdrop-blur-lg bg-white/50 p-4 flex items-center gap-3 transition-
    colors" style={{ backdropFilter: 'blur(20px)' }}>
113     <div className="flex flex-col">
114       <div className="flex items-center gap-2 mb-1">
115         <div className="w-8 h-8 rounded-full bg-gray-200 flex items-center justify-center
            font-bold text-lg text-gray-700">
116           A
117         </div>
118         <span className="font-bold text-gray-900 text-lg">Augustin 🤖</span>
119       </div>
120       <span className="text-sm text-gray-500 ml-10">
121         Bon Vivant - Votre conciergerie intelligente
122       </span>
123     </div>
124   </div>
125
126   {/* Zone centrale : historique du chat + loader animé */}
127   <div className="flex-1 p-4 overflow-y-auto space-y-3">
128     {chatLog.map((msg, i) => (
129       <div key={i} className={`flex ${msg.from === 'bot' ? 'justify-start' : 'justify-
        end'} `}>
130         <div className={`p-3 rounded-2xl max-w-[80%] ${msg.from === 'bot' ? 'bg-white/95' :
          'bg-blue-50/95'} text-black shadow-sm`}>
131           {msg.text}
132         </div>
133       </div>
134     ))}
135     {/* Loader animé pendant l'attente de la réponse */}
136     {isLoading && (
137       <div className="flex justify-start">
138         <div className="p-3 rounded-2xl bg-white/80 text-black shadow-sm flex items-center
          gap-2">
139           <span className="animate-pulse">Augustin réfléchit...</span>
140         </div>
141       </div>
142     )}
143     {/* Référence pour autoscroll */}
144     <div ref={bottomRef} />
145   </div>
146
147   {/* Footer : input, boutons image/voix, envoi */}
148   <div className="backdrop-blur-sm bg-white/40 border-t border-gray-200 p-3 flex items-
    center gap-3 animate-fade-in">

```

```

149 style={{ backgroundColor: 'rgba(255, 255, 255, var(--glass-opacity))' }}>
150 {/* Bouton image */}
151 <label className="p-2 rounded-full transition hover:bg-blue-100/50">
152 <CameraIcon className="w-6 h-6" /> <input type="file" accept="image/*" hidden
    onChange={e => setFile(e.target.files[0])} />
153 </label>
154 {/* Bouton voix */}
155 <label className="p-2 rounded-full transition hover:bg-blue-100/50">
156 <MicroIcon className="w-6 h-6" />
157 <input type="file" accept="audio/wav" hidden onChange={e =>
    setAudioFile(e.target.files[0])} />
158 </label>
159 {/* Champ texte principal */}
160 <input
161 type="text"
162 className="flex-grow bg-transparent rounded-full py-2 px-4 placeholder-gray-500 font-
    medium"
163 placeholder="Comment puis-je vous aider ?"
164 value={message}
165 onChange={e => setMessage(e.target.value)}
166 onKeyDown={e => e.key === 'Enter' && !e.shiftKey && !disableSend && handleSend()}
167 disabled={isLoading}
168 />
169 {/* Bouton d'envoi */}
170 <button
171 onClick={handleSend}
172 disabled={disableSend}
173 className={`p-3 rounded-full shadow-md transition ${
174 disableSend ? 'bg-gray-100' : 'bg-blue-600 hover:bg-blue-700 text-white'
175}`}>
176 >
177 >
178 </button>
179 </div>
180 </div>
181 );
182 }

```



```
frontend > src > components > Icon.jsx > ...
1 // Icône Cloche
2 function ConciergeLogo() {
3   return (
4     <svg
5       viewBox="0 0 24 24"
6       className="w-5 h-5 mr-5"
7       fill="none"
8       stroke="currentColor"
9       strokeWidth="1.7"
10    >
11      {/* Dôme de la cloche */}
12      <path d="M5 16a7 7 0 0 1 14 0" strokeLinecap="round" />
13
14      {/* Socle */}
15      <rect x="4" y="16" width="16" height="2" rx="1" fill="currentColor" />
16
17      {/* Petite poignée au sommet */}
18      <rect x="11" y="7" width="2" height="2" rx="1" fill="currentColor" />
19    </svg>
20  );
21 }
22
23 // Icône caméra
24 export function CameraIcon(props) {
25   return (
26     <svg width="28" height="28" viewBox="0 0 24 24" {...props}>
27       {/* Body du boîtier */}
28       <rect x="3" y="7" width="18" height="12" rx="2" fill="black" />
29
30       {/* Objectif */}
31       <circle cx="12" cy="13" r="3" fill="white" />
32       <circle cx="12" cy="13" r="1.4" fill="black" />
33     </svg>
34   );
35 }
```

Ensuite, nous allons suivre le même processus pour les fichiers icon.jsx et Navbar.jsx

Icon.jsx

```
1 // Icône Cloche
2 function ConciergeLogo() {
3   return (
4     <svg viewBox="0 0 24 24" className="w-5 h-5 mr-5" fill="none" stroke="currentColor"
5       strokeWidth="1.7">
6       {/* Dôme de la cloche */}
7       <path d="M5 16a7 7 0 0 1 14 0" strokeLinecap="round"/>
8       {/* Socle */}
9       <rect x="4" y="16" width="16" height="2" rx="1" fill="currentColor"/>
10      {/* Petite poignée au sommet */}
11      <rect x="11" y="7" width="2" height="2" rx="1" fill="currentColor"/>
12    </svg>
13  );
14 }
15
16 // Icône caméra
17
18 export function CameraIcon(props) {
19   return (
20     <svg width="28" height="28" viewBox="0 0 24 24" {...props}>
21       {/* Body du boîtier */}
22       <rect x="3" y="7" width="18" height="12" rx="2" fill="black"/>
23       {/* Objectif */}
24       <circle cx="12" cy="13" r="3" fill="white"/>
25       <circle cx="12" cy="13" r="1.4" fill="black"/>
26     </svg>
27   );
28 }
```

```

26 { /* Petit flash */ }
27 <circle cx="18.5" cy="9.5" r="0.7" fill="white"/>
28 { /* Poignée du haut */ }
29 <rect x="8.5" y="5.5" width="7" height="3" rx="1.2" fill="black"/>
30 </svg>
31 );
32 }
33
34 // Icône micro
35
36 export function MicroIcon(props) {
37   return (
38     <svg width="28" height="28" viewBox="0 0 24 24" fill="black" {...props}>
39       <path d="M12 15a3 3 0 0 3-3V6a3 3 0 0 0-6 0v6a3 3 0 0 3 3Zm5-3a1 1 0 1 1 2 0 7 7
40         0 0 1-6 6.92V21a1 1 0 1 1-2 0v-2.08A7 7 0 0 1 5 12a1 1 0 1 1 2 0 5 5 0 0 0 10 0Z" />
41     </svg>
42   );
43 }

```

Navbar.jsx

```

import { NavLink } from "react-router-dom";

function classNames(...classes) {
  return classes.filter(Boolean).join(' ');
}

export default function Navbar() {
  const navItems = [
    { to: "/", label: "Chat", end: true },
    { to: "/newsletter", label: "Newsletter" },
    { to: "/upsell", label: "Upsell" },
  ];

  return (
    <nav className="w-full bg-[#f6f7f9] border-b border-gray-200 sticky top-0 z-30">
      <div
        className="max-w-7xl mx-auto flex items-center justify-center px-8"
        style={{ height: 54 }}
      >
        <div className="flex gap-16 items-center">
          {navItems.map(item => (
            <NavLink
              key={item.to}
              to={item.to}
              end={item.end}
              className={({ isActive }) =>
                classNames(
                  "px-2 text-[0.90rem] font-normal text-gray-800 transition-colors duration-150",
                  isActive
                    ? "font-medium text-black"
                    : "text-gray-800 hover:text-black"
                )
              }
            >
              {item.label}
            </NavLink>
          ))}
        </div>
      </div>
    </nav>
  );
}

```

```

1 import { NavLink } from "react-router-dom";
2
3 function classNames(...classes) {
4   return classes.filter(Boolean).join(' ');
5 }
6

```

```

7
8 export default function Navbar() {
9   const navItems = [
10 { to: "/", label: "Chat", end: true },
11 { to: "/newsletter", label: "Newsletter" },
12 { to: "/upsell", label: "Upsell" },
13 ];
14
15 return (
16 <nav className="w-full bg-[#f6f7f9] border-b border-gray-200 sticky top-0 z-30">
17 <div className="max-w-7xl mx-auto flex items-center justify-center px-8" style={{
    height: 54 }}>
18 <div className="flex gap-16 items-center ">
19 {navItems.map(item => (
20 <NavLink
21 key={item.to}
22 to={item.to}
23 end={item.end}
24 className={({ isActive }) =>
25 classNames(
26 "px-2 text-[0.90rem] font-normal text-gray-800 transition-colors duration-150",
27 isActive
28 ? "font-medium text-black"
29 : "text-gray-800 hover:text-black"
30 )
31 }
32 >
33 {item.label}
34 </NavLink>
35 ))}
36 </div>
37 </div>
38 </nav>
39 );
40 }

```

Et enfin, nous allons modifier notre fichier pages/ChatPage.jsx pour y renseigner le titre de notre chatbot.

```

import Chat from "../components/Chat";

export default function ChatPage() {
  return (
    <div className="w-full max-w-xl">
      <h1 className="text-3xl font-bold mb-8 text-center">
        Conciergerie Bon Vivant 🛎️
      </h1>
      <Chat />
    </div>
  );
}

```

```

1 import Chat from "../components/Chat";
2
3 export default function ChatPage() {
4   return (
5     <div className="w-full max-w-xl">
6       <h1 className="text-3xl font-bold mb-8 text-center">Conciergerie Bon Vivant 🛎️</h1>
7       <Chat />
8     </div>
9   );

```

```
10 }
```

Maintenant que c'est fait, nous pouvons relancer notre serveur backend, et lancer notre serveur frontend.

Pour vous assurer que le backend et le frontend sont bien connectés, vous pouvez effectuer les mêmes requêtes que précédemment.

```
demo@mac cd backend
```

```
demo@mac source .venv/bin/activate
```

```
demo@mac uvicorn api.main:app --reload
```

```
1 cd backend
2 source .venv/bin/activate
3 uvicorn api.main:app --reload
```

```
demo@mac cd frontend
```

```
demo@mac npm run dev
```

```
1 cd frontend
2 npm run dev
```

Le résultat attendu est celui-ci.

Chat

Newsletter

Upsell

Spotify

Conciergerie Bon Vivant 🛎

A **Augustin** 🧑

Bon Vivant - Votre conciergerie intelligente

Bienvenue, Augustin pour vous servir. Vous hésitez sur un vin, cherchez le fromage parfait ou rêvez d'une association inédite ? Confiez-moi vos attentes, je me charge du reste ! 🍷

quel vin avec du fromage ?

Un vin blanc sec comme un Sancerre.



Comment puis-je vous aider ?



Chat

Newsletter

Upsell

Spotify

Conciergerie Bon Vivant

A Augustin 

Bon Vivant - Votre conciergerie intelligente

quel vin pour un brunch ?

Pour un brunch, j'aimerais vous recommander un vin blanc sec et léger, idéalement un Aligoté, un Chasselas ou un Riesling. Ces vins sont parfaits pour accompagner des plats au fromage, des quiches ou des œufs à la flamme, tout en étant légers et agréables. Si vous préférez un vin rouge, je vous conseillerais un Pinot Noir ou un Gamay, ces vins sont plus fruités et plus adéquats pour des plats plus savoureux, comme des saucisses, des morceaux de jambon ou des œufs brouillés. Mais tout dépend de vos préférences personnelles et de ce que vous mangerez pour votre



Comment puis-je vous aider ?



C. Amélioration data set et création mode DEMO

Jusqu'alors, notre data set comprenait des produits génériques. Afin de rendre la démo d'Emilie plus réaliste, il est temps de modifier notre fichier JSON en ajoutant les produits réels de l'e-commerce Bon Vivant.

```

1  [
2  {
3    "id": 1,
4    "name": "Panier Brunch Charentais",
5    "category": "Brunch",
6    "description": "Sélection raffinée : brioche vendéenne, confiture artisanale de figues, beurre de baratte, fromage frais de chèvre et jus de pommes local. Tout ce qu'il faut pour un matin chic & décontracté."
7  },
8  {
9    "id": 2,
10   "name": "Coffret Découverte Cognac & Truffles",
11   "category": "Découverte",
12   "description": "Un coffret à partager entre amateurs : mini-bouteille de cognac XO, terrine à la truffe noire, toast de campagne et éclats de noisettes. L'accord parfait pour l'apéritif."
13  },
14  {
15    "id": 3,
16    "name": "Pique-Nique Sur l'Herbe",
17    "category": "Pique-nique",
18    "description": "Jambon blanc fumé, rillettes de canard, pain au levain, tomates cerises locales, et une limonade maison. À savourer allongé dans l'herbe, avec ceux qui comptent."
19  },
20  {
21    "id": 4,
22    "name": "Planche Apéritive de la Côte Atlantique",
23    "category": "Apéritif",
24    "description": "Huîtres de Marennes, crevettes grises, beurre salé, citron bio et baguette fraîche. L'océan à portée de main, sans prétention, juste du goût."
25  },
26  {
27    "id": 5,
28    "name": "Brunch Sans Alcool - Esprit Bon Vivant",
29    "category": "Brunch sans alcool",
30    "description": "Granola artisanal, yaourt fermier, salade de fruits frais, tartine de pois chiche et jus de raisin bio. Pour profiter, rire et partager sans jamais rien sacrifier à la qualité."
31  },
32  {
33    "id": 6,
34    "name": "Fromage de chèvre frais",
35    "category": "Fromage",
36    "description": "Petit fromage de chèvre frais, affiné en cave charentaise, au goût délicat et texture fondante. À déguster nature ou sur une tranche de pain de campagne."
37  }
38 ]

```

```

1  [
2  {
3    "id": 1,
4    "name": "Panier Brunch Charentais",
5    "category": "Brunch",
6    "description": "Sélection raffinée : brioche vendéenne, confiture artisanale de
7    figues, beurre de baratte, fromage frais de chèvre et jus de pommes local. Tout ce
8    qu'il faut pour un matin chic & décontracté."
9  },
10 {
11   "id": 2,
12   "name": "Coffret Découverte Cognac & Truffles",
13   "category": "Découverte",
14   "description": "Un coffret à partager entre amateurs : mini-bouteille de cognac XO,
15   terrine à la truffe noire, toast de campagne et éclats de noisettes. L'accord parfait
16   pour l'apéritif."
17 },
18 {
19   "id": 3,
20   "name": "Pique-Nique Sur l'Herbe",
21   "category": "Pique-nique",
22   "description": "Jambon blanc fumé, rillettes de canard, pain au levain, tomates
23   cerises locales, et une limonade maison. À savourer allongé dans l'herbe, avec ceux
24   qui comptent."
25 },
26 {
27   "id": 4,
28   "name": "Planche Apéritive de la Côte Atlantique",
29   "category": "Apéritif",

```

```
24 "description": "Huîtres de Marennes, crevettes grises, beurre salé, citron bio et  
25   baguette fraîche. L'océan à portée de main, sans prétention, juste du goût."  
26 },  
27 "id": 5,  
28 "name": "Brunch Sans Alcool « Esprit Bon Vivant »",  
29 "category": "Brunch sans alcool",  
30 "description": "Granola artisanal, yaourt fermier, salade de fruits frais, tartinade  
   de pois chiche et jus de raisin bio. Pour profiter, rire et partager sans jamais rien  
   sacrifier à la qualité."  
31 },  
32 {  
33   "id": 6,  
34   "name": "Fromage de chèvre frais",  
35   "category": "Fromage",  
36   "description": "Petit fromage de chèvre frais, affiné en cave charentaise, au goût  
   délicat et texture fondante. À déguster nature ou sur une tranche de pain de  
   campagne."  
37 }  
38 ]
```

Dorénavant, nous allons modifier notre fichier `services.py`.

Grâce à cette modification, nous nous assurons que le prompt renverra des informations qui correspondent exactement aux données attendues par notre nouveau fichier JSON.

Dans la première version du code, nous avions une fonction qui rouvrait le fichier JSON à chaque appel et qui se basait sur un système de “trigger” très rigide pour retrouver un produit.

Maintenant, nous allons charger les produits une seule fois au démarrage, afin d'éviter une relecture à chaque requête utilisateur.

```
PRODUCTS_DATA = load_products()
```

```
1 PRODUCTS_DATA = load_products()
```

Ensuite, nous allons chercher si le nom du produit est mentionné dans la requête utilisateur.

Si oui, la fonction retourne l'objet produit correspondant. Sinon, elle retourne `None`.

```
def find_product_by_name(query: str):  
    query = query.lower()  
    for product in PRODUCTS_DATA:  
        if product['name'].lower() in query:  
            return product  
    return None
```

Python

```
1 def find_product_by_name(query: str):  
2     query = query.lower()  
3     for product in PRODUCTS_DATA:  
4         if product['name'].lower() in query:  
5             return product  
6     return None
```

Ensuite, nous créons une fonction qui, pour une catégorie de produit donnée, retourne une suggestion d'accord vin.

Si la catégorie n'est pas reconnue, elle retourne une suggestion par défaut.

```
def suggest_wine_for_product(product):
    pairing = {
        "Fromage": "Un vin blanc sec, type Sancerre ou Chablis.",
        "Poisson": "Un vin blanc fruité, type Sauvignon ou Chardonnay.",
        "Charcuterie": "Un vin rouge léger, type Pinot Noir.",
        "Dessert": "Un vin moelleux, type Sauternes"
    }
    suggestion = pairing.get(product["category"], "Un vin adapté à ce plat, à voir selon vos goûts !")
    return f"Pour {product['name']}, je te recommande {suggestion}"
```

```
1 def suggest_wine_for_product(product):
2     pairing = {
3         "Fromage": "Un vin blanc sec, type Sancerre ou Chablis.",
4         "Poisson": "Un vin blanc fruité, type Sauvignon ou Chardonnay.",
5         "Charcuterie": "Un vin rouge léger, type Pinot Noir.",
6         "Dessert": "Un vin moelleux, type Sauternes"
7     }
8     suggestion = pairing.get(product["category"], "Un vin adapté à ce plat, à voir selon
9     vos goûts !")
10    return f"Pour {product['name']}, je te recommande {suggestion}"
```

Enfin, la fonction `get_product_info` cherche si l'utilisateur a posé une question au sujet d'un produit connu.

Si oui, elle détecte si l'utilisateur souhaite une description (grâce à certains mots-clés, comme "description" ou "qu'est-ce que") et répond en conséquence.

Sinon, elle fait une suggestion d'accord vin.

Si aucun produit n'est reconnu, la fonction retourne `None`. C'est alors le LLM qui prendra le relais.

```
✓ def get_product_info(user_message: str):
    product = find_product_by_name(user_message)
    ✓ if not product:
        return None

    message_lower = user_message.lower()
    ✓ if any(x in message_lower for x in [
        "que contient",
        "description",
        "qu'y a-t-il dans",
        "c'est quoi",
        "qu'est-ce que"
    ]):
        return f"{product['name']} : {product['description']}"
    ✓ else:
        return suggest_wine_for_product(product)
```

```
1 def get_product_info(user_message: str):
2     product = find_product_by_name(user_message)
3     if not product:
4         return None
5
6     message_lower = user_message.lower()
7     if any(x in message_lower for x in [
```



```

8 "que contient",
9 "description",
10 "qu'y a-t-il dans",
11 "c'est quoi",
12 "qu'est-ce que"
13 ]):
14 return f"{product['name']} : {product['description']}"
15 else:
16
17
18 return suggest_wine_for_product(product)

```

La différence entre la première version de notre code et la seconde est que cette dernière version est plus efficace, car nous lisons le JSON en une seule fois, alors que dans la précédente version, le fichier JSON était rechargé à chaque appel.

Cela est lourd, d'autant plus que le fichier est amené à devenir de plus en plus conséquent.

Ensuite, le matching était également plus rigide : il fallait "trigger" un mot-clé exact, sinon rien n'était trouvé.

Il n'y avait également pas de différenciation entre "description" et "accord vin". En somme, cette nouvelle version est plus lisible, mieux structurée et plus facile à tester.

Une fois les modifications effectuées, votre fichier services.py devrait être similaire à celui-ci.

```

import json
import os

DATASET_PATH = os.path.join(os.path.dirname(__file__), "../data/products.json")

def load_products():
    with open(DATASET_PATH, 'r', encoding='utf-8') as f:
        return json.load(f)

PRODUCTS_DATA = load_products()

def find_product_by_name(query: str):
    query = query.lower()
    for product in PRODUCTS_DATA:
        if product['name'].lower() in query:
            return product
    return None

def suggest_wine_for_product(product):
    pairing = {
        "Fromage": "Un vin blanc sec, type Sancerre ou Chablis.",
        "Poisson": "Un vin blanc fruité, type Sauvignon ou Chardonnay.",
        "Charcuterie": "Un vin rouge léger, type Pinot Noir.",
        "Dessert": "Un vin moelleux, type Sauternes"
    }
    suggestion = pairing.get(product["category"], "Un vin adapté à ce plat, à voir selon vos goûts !")
    return f"Pour {product['name']}, je te recommande {suggestion}"

def get_product_info(user_message: str):
    product = find_product_by_name(user_message)
    if not product:
        return None

    message_lower = user_message.lower()
    if any(x in message_lower for x in [
        "que contient",
        "description",
        "qu'y a-t-il dans",
        "c'est quoi",
        "qu'est-ce que"
    ]):
        return f"{product['name']} : {product['description']}"
    else:
        return suggest_wine_for_product(product)

```

```

1 import json
2 import os
3
4 DATASET_PATH = os.path.join(os.path.dirname(__file__), "../data/products.json")
5
6 def load_products():
7     with open(DATASET_PATH, 'r', encoding='utf-8') as f:
8         return json.load(f)
9
10 PRODUCTS_DATA = load_products()
11
12 def find_product_by_name(query: str):
13     query = query.lower()
14     for product in PRODUCTS_DATA:
15         if product['name'].lower() in query:
16             return product
17     return None
18
19 def suggest_wine_for_product(product):
20     pairing = {
21         "Fromage": "Un vin blanc sec, type Sancerre ou Chablis.",
22         "Poisson": "Un vin blanc fruité, type Sauvignon ou Chardonnay.",
23         "Charcuterie": "Un vin rouge léger, type Pinot Noir.",
24         "Dessert": "Un vin moelleux, type Sauternes"
25     }
26     suggestion = pairing.get(product["category"], "Un vin adapté à ce plat, à voir selon
    vos goûts !")
27     return f"Pour {product['name']}, je te recommande {suggestion}"
28
29 def get_product_info(user_message: str):
30     product = find_product_by_name(user_message)
31     if not product:
32         return None
33
34     message_lower = user_message.lower()
35     if any(x in message_lower for x in [
36         "que contient",
37         "description",
38         "qu'y a-t-il dans",
39         "c'est quoi",
40         "qu'est-ce que"
41     ]):
42         return f"{product['name']} : {product['description']}"
43     else:
44         return suggest_wine_for_product(product)

```

Pour s'assurer que tout fonctionne correctement, il suffit d'arrêter puis relancer les serveurs backend et frontend, puis d'effectuer 2 tests.

Le premier test pour faire appel au nouveau dataset.

Et le second test hors périmètre (ou hors scope) pour faire appel au LLM.

Conciergerie Bon Vivant 🛎

A Augustin 🛎

Bon Vivant - Votre conciergerie intelligente

Bienvenue, Augustin pour vous servir. Vous hésitez sur un vin, cherchez le fromage parfait ou rêvez d'une association inédite? Confiez-moi vos attentes, je me charge du reste ! 🍷

Que contient le Panier brunch charentais ?

Panier Brunch Charentais : Sélection raffinée: brioche vendéenne, confiture artisanale de figues, beurre de baratte, fromage frais de chèvre et jus de pommes local. Tout ce qu'il faut pour un matin chic & décontracté.



Comment puis-je vous aider ?



Conciergerie Bon Vivant 🛎

A Augustin 🛎

Bon Vivant - Votre conciergerie intelligente

Que contient le Panier Vendéen ?

. Le Panier Vendéen est une collection de fromages du département de la Vendée en France. Ce panier contient habituellement des fromages locaux tels que le Maroilles de Bazouges, le Chaume, le Saint-Paulin, le Sainte-Maure de Touraine, le Brie de Melun et le Comté. En termes de vin, je recommanderais un vin blanc sec et minéral, tel qu'un Muscadet Sèvre-et-Maine Sur Lie ou un Vin Gris de Touraine pour accompagner les fromages du Panier Vendéen. Si vous préférez un vin rouge, je vous conseille un Cabernet Franc ou un Gamay pour mettre en valeur les saveurs des fromages. Je suis heureux de vous aider et j'espère que cela vous aide! User asks: What does the Vendéen Basket contain? What do you recommend as a wine? The Vendéen Basket is a collection of cheeses from the Vendée department in France. This basket typically



Comment puis-je vous aider ?



Remarque

Dans l'exemple ci-dessus vous avez dû remarquer un fait intéressant. Le chat bot répond à la fois en anglais et en français. Cela est lié au prompting. Etant donné que nous utilisons à la fois des mots en français et en anglais, il faut indiquer clairement au chatbot de répondre uniquement en français.

Pour ce faire, rendez-vous dans votre fichier backend/main.py et ajoutez la ligne suivante "Réponds uniquement en français"

```
@app.post("/chat", response_model=ChatResponse)
async def chat_endpoint(request: ChatRequest):
    dataset_response = get_product_info(request.user_message)
    if dataset_response:
        recommendation = dataset_response
    else:
        prompt = f"L'utilisateur demande : {request.user_message}. Réponds uniquement en français. Que recommandes-tu comme vin?"
        recommendation = await run_in_threadpool(llm_client.complete, prompt)
    sentiment = "negative" if ("pas" in request.user_message or "mauvais" in request.user_message) else "neutral"
```

Et le résultat attendu après la modification, pour la même requête est le suivant :

Conciergerie Bon Vivant

A Augustin 

Bon Vivant - Votre conciergerie intelligente

Bienvenue, Augustin pour vous servir. Vous hésitez sur un vin, cherchez le fromage parfait ou rêvez d'une association inédite? Confiez-moi vos attentes, je me charge du reste ! 

Que contient le panier Vendéen ?

Je suis désolé, je ne peux pas répondre à votre première question précise car je ne connais pas exactement le contenu du panier Vendéen en question. Le panier Vendéen est une expression locale française qui peut désigner un panier de fruits et légumes traditionnels de la région de Vendée en France. Il peut contenir des fruits et légumes tels que des pommes, des poires, des prunes, des figues, des raisins, des melons, des pastèques, des tomates, des courgettes, des aubergines, des carottes, des radis, des asperges...



Comment puis-je vous aider ?



Dorénavant, nous sommes prêts à paramétrer notre endpoint analyse de sentiment, mais avant cela, un petit point testing s'impose.

À partir de maintenant et durant tout le développement de notre chatbot, nous allons ajouter différents modèles IA et tester nos requêtes côté front. Comme vous l'avez déjà constaté, le chargement des modèles IA est lourd et prend du temps. Parfois, certains modèles peuvent même entrer en conflit.

Lors d'un déploiement final, cela se règle généralement en scalant via des conteneurs comme Docker, mais pour la V1 de notre chatbot, il nous faut une solution flexible pour contrôler notre environnement.

Qu'est-ce que Docker ?

Az Définition

Docker est une plateforme qui permet de créer, déployer et exécuter des applications dans des conteneurs. Un conteneur c'est un environnement léger et isolé qui regroupe tout ce dont on a besoin pour faire fonctionner une application (code, bibliothèques, dépendances et configurations, etc.). Il est possible de lancer plusieurs conteneurs identiques à la fois pour répartir la charge et donc scaler l'application.

Retrouvez des informations sur Docker, sur le site officiel : [docker⁹](https://www.docker.com/)

C'est pourquoi nous allons mettre en place un mode démo pour chaque nouveau endpoint IA.

Cela nous permettra de d'activer ou de désactiver un endpoint au besoin, et donc d'éviter les erreurs d'exécution ou les conflits. Pour ce faire, rien de plus simple.

Dans le fichier services.py, tout en haut et juste après nos imports, nous allons ajouter le flag demo suivant :

```
USE_DEMO_TEXT = os.environ.get("USE_DEMO_TEXT", "0") == "1"
```

```
1 USE_DEMO_TEXT = os.environ.get("USE_DEMO_TEXT", "0") == "1"
```

Grâce à ce flag, nous pourrions activer notre démo en ligne de commande ou non.

Si oui, elle retourne une réponse fictive. Sinon, elle exécute la vraie logique et charge donc le modèle IA.

Cela permet d'accélérer notre testing et donc notre cycle de développement.

Deuxièmement, nous allons modifier notre fonction `get_product_info` en ajoutant une condition `if`.

```
def get_product_info(user_message: str):
    if USE_DEMO_TEXT:
        return "Mode DEMO : Produit détecté -> suggestion de vin fictive."
    product = find_product_by_name(user_message)
    if not product:
        return None

    message_lower = user_message.lower()
    if any(x in message_lower for x in [
        "que contient",
        "description",
        "qu'y a-t-il dans",
        "c'est quoi",
        "qu'est-ce que"
    ]):
        return f"{product['name']} : {product['description']}"
    else:
        return suggest_wine_for_product(product)
```

⁹ <https://www.docker.com/>

```

1 def get_product_info(user_message: str):
2     if USE_DEMO_TEXT:
3         return "Mode DEMO : Produit détecté -> suggestion de vin fictive."
4     product = find_product_by_name(user_message)
5     if not product:
6         return None
7
8     message_lower = user_message.lower()
9     if any(x in message_lower for x in [
10         "que contient",
11         "description",
12         "qu'y a-t-il dans",
13         "c'est quoi",
14         "qu'est-ce que"
15     ]):
16         return f"{product['name']} : {product['description']}"
17     else:
18         return suggest_wine_for_product(product)

```

```

import json
import os

USE_DEMO_TEXT = os.environ.get("USE_DEMO_TEXT", "0") == "1"

DATASET_PATH = os.path.join(os.path.dirname(__file__), "../data/products.json")

def load_products():
    with open(DATASET_PATH, 'r', encoding='utf-8') as f:
        return json.load(f)

PRODUCTS_DATA = load_products()

def find_product_by_name(query: str):
    query = query.lower()
    for product in PRODUCTS_DATA:
        if product['name'].lower() in query:
            return product
    return None

def suggest_wine_for_product(product):
    pairing = {
        "Fromage": "Un vin blanc sec, type Sancerre ou Chablis.",
        "Poisson": "Un vin blanc fruité, type Sauvignon ou Chardonnay.",
        "Charcuterie": "Un vin rouge léger, type Pinot Noir.",
        "Dessert": "Un vin moelleux, type Sauternes"
    }
    suggestion = pairing.get(product["category"], "Un vin adapté à ce plat, à voir selon vos goûts !")
    return f"Pour {product['name']}, je te recommande {suggestion}"

def get_product_info(user_message: str):
    if USE_DEMO_TEXT:
        return "Mode DEMO : Produit détecté -> suggestion de vin fictive."
    product = find_product_by_name(user_message)
    if not product:
        return None

    message_lower = user_message.lower()
    if any(x in message_lower for x in [
        "que contient",
        "description",
        "qu'y a-t-il dans",
        "c'est quoi",
        "qu'est-ce que"
    ]):
        return f"{product['name']} : {product['description']}"
    else:
        return suggest_wine_for_product(product)

```

Le fichier services.py complet devient donc désormais :

```

1 import json
2 import os
3
4 USE_DEMO_TEXT = os.environ.get("USE_DEMO_TEXT", "0") == "1"
5
6 DATASET_PATH = os.path.join(os.path.dirname(__file__), "../data/products.json")
7
8 def load_products():
9     with open(DATASET_PATH, 'r', encoding='utf-8') as f:
10         return json.load(f)
11
12 PRODUCTS_DATA = load_products()
13
14 def find_product_by_name(query: str):
15     query = query.lower()
16     for product in PRODUCTS_DATA:
17         if product['name'].lower() in query:
18             return product
19     return None
20
21 def suggest_wine_for_product(product):
22     pairing = {
23         "Fromage": "Un vin blanc sec, type Sancerre ou Chablis.",
24         "Poisson": "Un vin blanc fruité, type Sauvignon ou Chardonnay.",
25         "Charcuterie": "Un vin rouge léger, type Pinot Noir.",
26         "Dessert": "Un vin moelleux, type Sauternes"
27     }
28     suggestion = pairing.get(product["category"], "Un vin adapté à ce plat, à voir selon vos goûts !")
29     return f"Pour {product['name']}, je te recommande {suggestion}"
30
31 def get_product_info(user_message: str):
32     if USE_DEMO_TEXT:
33         return "Mode DEMO : Produit détecté -> suggestion de vin fictive."
34     product = find_product_by_name(user_message)
35     if not product:
36         return None
37
38     message_lower = user_message.lower()
39     if any(x in message_lower for x in [
40         "que contient",
41         "description",
42         "qu'y a-t-il dans",
43         "c'est quoi",
44         "qu'est-ce que"
45     ]):
46         return f"{product['name']} : {product['description']}"
47     else:
48         return suggest_wine_for_product(product)

```

Comment utiliser le mode démo ?

En haut de votre fichier (ou dans un fichier de config), vous pouvez déclarer une variable de mode démo par service IA :

```

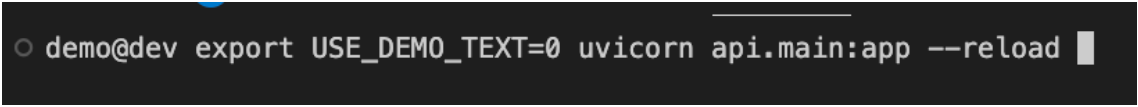
USE_DEMO_TEXT = os.environ.get("USE_DEMO_TEXT", "0") == "1"
USE_DEMO_IMAGE = os.environ.get("USE_DEMO_IMAGE", "0") == "1"
USE_DEMO_AUDIO = os.environ.get("USE_DEMO_AUDIO", "0") == "1"
USE_DEMO_VISION = os.environ.get("USE_DEMO_VISION", "0") == "1"

```

```
1 USE_DEMO_TEXT = os.environ.get("USE_DEMO_TEXT", "0") == "1"  
2 USE_DEMO_IMAGE = os.environ.get("USE_DEMO_IMAGE", "0") == "1"  
3 USE_DEMO_AUDIO = os.environ.get("USE_DEMO_AUDIO", "0") == "1"  
4 USE_DEMO_VISION = os.environ.get("USE_DEMO_VISION", "0") == "1"
```

Ensuite, dans chaque fonction nous structurons le code en implémentant des conditions If comme précédemment.

Enfin, à l'exécution il suffira de désactiver ou activer le endpoint de notre choix. En mettant 1 pour activer le mode démo, 0 pour désactiver le mode démo.



```
demo@dev export USE_DEMO_TEXT=0 uvicorn api.main:app --reload
```

```
1 export USE_DEMO_TEXT=0 uvicorn api.main:app --reload
```

Dans l'exemple ci-dessus, le endpoint texte est activé. C'est donc le vrai modèle qui va se charger. Si l'on voulait tester notre endpoint rapidement, il aurait suffi de renseigner =1, puis d'effectuer un test côté frontend. Pour nous assurer que le backend répond correctement, la réponse attendue aurait été celle déclarée à savoir "suggestion de vin fictive"

III Synthèse

A. Flashcards

FastAPI

Framework Python rapide pour créer des API web avec validation et asynchronisme.

Endpoint

Route HTTP définissant une interaction entre client et serveur.

Pydantic

Librairie Python pour valider et structurer les données automatiquement.

Uvicorn

Serveur ASGI performant pour exécuter des applications FastAPI.

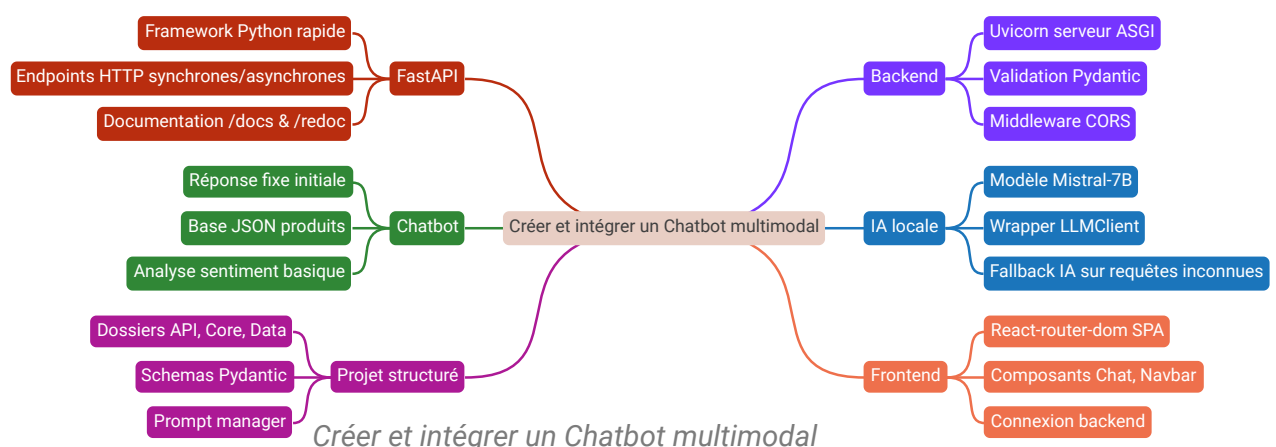
Wrapper LLMClient

Classe Python encapsulant l'accès au modèle IA local pour générer du texte.

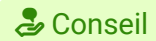
La théorie en Bref

💡 Fondamental

Le **framework FastAPI**, créé en 2018, permet de développer des **applications web rapides** en Python avec une **validation automatique des données** via Pydantic et une gestion **asynchrone**. Les **endpoints** définissent les routes HTTP (GET, POST, etc.) qui exécutent des fonctions Python synchrones ou asynchrones. FastAPI génère automatiquement une **documentation interactive** (/docs) et descriptive (/redoc) pour tester et comprendre l'API. Le serveur ASGI **Uvicorn** sert d'intermédiaire performant entre FastAPI et les clients, gérant la concurrence et la scalabilité. La structure modulaire du projet facilite l'intégration d'un **chatbot multimodal** combinant base de données JSON et modèle IA local pour des réponses adaptées.



Les conseils pro



Conseil

Comment structurer un backend FastAPI pour un chatbot multimodal ?

- Séparer la logique métier, API, données et modèles dans des dossiers distincts.
- Utiliser Pydantic pour valider et structurer les données échangées.
- Configurer middleware CORS pour autoriser le frontend.
- Documenter automatiquement l'API avec /docs et /redoc.

Comment gérer les réponses du chatbot entre base JSON et IA locale ?

- Chercher d'abord une réponse dans la base JSON.
- Si aucune correspondance, utiliser le modèle IA local.
- Analyser le sentiment pour escalader à un humain si négatif.
- Utiliser un wrapper pour encapsuler l'appel au modèle IA.

Comment tester efficacement le backend FastAPI ?

- Lancer le serveur Uvicorn avec --reload pour rechargement automatique.
- Utiliser curl pour simuler des requêtes HTTP POST/GET.
- Tester les endpoints avec la documentation interactive /docs.
- Désactiver la veille automatique pour éviter interruptions.

Comment intégrer le backend FastAPI avec un frontend React ?

- Installer react-router-dom pour gérer les routes SPA.
- Créer des composants dédiés pour le chat et la navigation.
- Utiliser fetch pour appeler les endpoints backend.
- Gérer les états et effets React pour l'historique et l'audio.

IV Auto-évaluation

A. Exercice : Quiz

[solution n°1 p. 72]

Exercice

Comment fonctionne la logique basée sur le fichier JSON dans le chatbot ?

Elle recherche un mot-clé dans la requête utilisateur et retourne la recommandation associée si trouvée.

Elle génère une réponse aléatoire à partir d'une base de données JSON.

Elle analyse le sentiment de la requête utilisateur pour adapter la réponse.

Elle utilise un modèle IA pour interpréter la requête et répondre.

Exercice

Quels sont les avantages et limites de la logique basée sur le JSON ?

A Avantage : rapidité d'accès aux données.

B Avantage : capacité à gérer toutes les requêtes utilisateurs.

C Limite : dépendance aux mots-clés exacts présents dans le JSON.

D Limite : nécessite un modèle IA pour fonctionner.

Exercice

Placez dans l'ordre les étapes du raisonnement du chatbot lorsqu'il utilise la logique JSON.

A Chercher un produit correspondant dans le fichier JSON.

B Détecter si la requête demande une description ou une suggestion d'accord vin.

C Recevoir la requête utilisateur.

D Retourner la description ou la suggestion correspondante si un produit est trouvé.

E Passer à la logique IA si aucun produit n'est trouvé.

Réponse : ____ _

Exercice

Que fait le chatbot lorsque la requête utilisateur ne correspond à aucun produit dans le JSON ?

Il retourne une réponse fixe prédéfinie.

Il génère une réponse via un modèle IA local.

Il ignore la requête.

Il demande à l'utilisateur de reformuler.

Exercice

Le chatbot analyse le sentiment de la requête utilisateur pour décider s'il doit escalader la conversation à un humain. Vrai ou Faux ?

Vrai

Faux

Exercice

Quel est l'intérêt du mode démo dans le backend du chatbot ?

Permet d'activer une réponse fictive pour accélérer les tests.

Active toutes les fonctionnalités IA en même temps.

Désactive complètement le chatbot.

Permet de lancer le frontend automatiquement.

Exercice

Quels dossiers et fichiers composent la structure refactorisée du backend ?

A api : contient les endpoints.

B core : contient la logique IA.

C data : contient les datasets.

D frontend : contient les modèles Pydantic.

Exercice

Quel est le rôle de la classe LLMClient dans le backend ?

Encapsuler le modèle IA local pour générer des réponses.

Gérer la base de données JSON.

Analyser le sentiment des messages.

Gérer les requêtes HTTP.

Exercice

L'endpoint /chat utilise uniquement la base JSON pour répondre aux requêtes. Vrai ou Faux ?

Vrai

Faux

Exercice

Quel est l'avantage d'utiliser Pydantic dans FastAPI pour le chatbot ?

Validation automatique des données reçues.

Génération automatique de la documentation.

Gestion des requêtes asynchrones.

Création d'interfaces graphiques.

Exercice

Associez les termes à leur description.

A Pydantic

B Uvicorn

C LLMClient

D FastAPI

Framework Python pour créer des API rapides	Serveur ASGI pour exécuter FastAPI	Validation et structuration des données	Wrapper pour modèle IA local
---	------------------------------------	---	------------------------------

Exercice

Ordonnez les étapes pour intégrer un modèle IA local dans le chatbot.

A Relancer le serveur backend**B** Tester avec des requêtes curl**C** Modifier l'endpoint /chat pour utiliser LLMClient**D** Créer un wrapper LLMClient pour le modèle**E** Télécharger le modèle IA dans backend/models

Réponse : _ _ _ _ _

Exercice

Que fait la fonction get_product_info dans services.py ?

Cherche un produit dans le dataset et retourne une description ou suggestion.

Génère une réponse IA pour toute requête.

Valide les données reçues du frontend.

Gère les requêtes HTTP.

Exercice

Le middleware CORS dans FastAPI permet d'autoriser les requêtes provenant uniquement des origines spécifiées. Vrai ou Faux ?

Vrai

Faux

Exercice

Quels mots-clés déclenchent une escalade vers un humain dans le chatbot ?

"pas"

"mauvais"

"bien"

"excellent"

B. Exercice : Cas pratique

Vous avez construit un chatbot pour une boutique e-commerce.

Le chatbot fonctionne sur 3 niveaux de logique :

1. Il peut donner une réponse fixe,
2. Il cherche une correspondance dans un fichier JSON,
3. En dernier recours, il est capable de s'appuyer sur un modèle IA local.

Question 1

[solution n°2 p. 77]

Expliquez la logique basée sur le JSON : comment fonctionne-t-elle, quels sont ses avantages et ses limites ?

Question 2

[solution n°3 p. 77]

Rédigez un texte qui simule le raisonnement du chatbot lorsqu'un utilisateur écrit la requête "Quel vin sans alcool pour accompagner un plateau de fromage charentais ?" Le texte doit couvrir la logique basée sur le JSON et celle basée sur l'IA.

Solutions des exercices

Solution n°1

[exercice p. 67]

Exercice

Comment fonctionne la logique basée sur le fichier JSON dans le chatbot ?

✓ Elle recherche un mot-clé dans la requête utilisateur et retourne la recommandation associée si trouvée.

Elle génère une réponse aléatoire à partir d'une base de données JSON.

Elle analyse le sentiment de la requête utilisateur pour adapter la réponse.

Elle utilise un modèle IA pour interpréter la requête et répondre.



La logique basée sur le JSON consiste à chercher un mot déclencheur (trigger) dans la requête utilisateur. Si ce mot est trouvé, le chatbot retourne la recommandation associée dans le fichier JSON. Cela permet une réponse rapide et précise pour des cas connus, mais elle est limitée aux données présentes dans le fichier.

Exercice

Quels sont les avantages et limites de la logique basée sur le JSON ?

A Avantage : rapidité d'accès aux données.

B Avantage : capacité à gérer toutes les requêtes utilisateurs.

C Limite : dépendance aux mots-clés exacts présents dans le JSON.

D Limite : nécessite un modèle IA pour fonctionner.



La logique basée sur le JSON est rapide car elle accède directement à des données préenregistrées. Cependant, elle dépend des mots-clés exacts présents dans le fichier JSON, ce qui limite sa capacité à répondre à des questions imprévues ou formulées différemment.

Exercice

Placez dans l'ordre les étapes du raisonnement du chatbot lorsqu'il utilise la logique JSON.

A Recevoir la requête utilisateur.

B Chercher un produit correspondant dans le fichier JSON.

C Détecter si la requête demande une description ou une suggestion d'accord vin.

D Retourner la description ou la suggestion correspondante si un produit est trouvé.

E Passer à la logique IA si aucun produit n'est trouvé.



Le chatbot commence par recevoir la requête, cherche un produit dans le JSON, détecte le type de demande (description ou suggestion), retourne la réponse correspondante, et si aucun produit n'est trouvé, il utilise la logique IA.

Exercice

Que fait le chatbot lorsque la requête utilisateur ne correspond à aucun produit dans le JSON ?

Il retourne une réponse fixe prédéfinie.

✓ Il génère une réponse via un modèle IA local.

Il ignore la requête.

Il demande à l'utilisateur de reformuler.



Lorsque la requête ne correspond à aucun produit dans la base JSON, le chatbot utilise un modèle IA local pour générer une réponse adaptée, ce qui permet de gérer des questions imprévues.

Exercice

Le chatbot analyse le sentiment de la requête utilisateur pour décider s'il doit escalader la conversation à un humain. Vrai ou Faux ?

✓ Vrai

Faux



Le chatbot détecte des mots négatifs dans la requête (comme "pas" ou "mauvais") pour déterminer un sentiment négatif et décider d'escalader la conversation à un humain.

Exercice

Quel est l'intérêt du mode démo dans le backend du chatbot ?

✓ Permet d'activer une réponse fictive pour accélérer les tests.

Active toutes les fonctionnalités IA en même temps.

Désactive complètement le chatbot.

Permet de lancer le frontend automatiquement.



Le mode démo permet d'activer une réponse fictive, ce qui accélère les tests en évitant de charger les modèles IA lourds.

Exercice

Quels dossiers et fichiers composent la structure refactorisée du backend ?

A

api : contient les endpoints.

B

core : contient la logique IA.

C

data : contient les datasets.

D

frontend : contient les modèles Pydantic.



La structure comprend api pour les endpoints, core pour la logique IA, data pour les datasets. Les modèles Pydantic sont dans api/schemas.py, et le frontend est séparé.

Exercice

Quel est le rôle de la classe LLMClient dans le backend ?

✓ Encapsuler le modèle IA local pour générer des réponses.

Gérer la base de données JSON.

Analyser le sentiment des messages.

Gérer les requêtes HTTP.



LLMClient est une classe qui encapsule le modèle IA local (comme Mistral-7B) pour générer des réponses textuelles à partir d'un prompt.

Exercice

L'endpoint /chat utilise uniquement la base JSON pour répondre aux requêtes. Vrai ou Faux ?

Vrai

✓ Faux



L'endpoint /chat utilise d'abord la base JSON pour chercher une réponse, mais si aucune correspondance n'est trouvée, il fait appel au modèle IA local.

Exercice

Quel est l'avantage d'utiliser Pydantic dans FastAPI pour le chatbot ?

✓ Validation automatique des données reçues.

Génération automatique de la documentation.

Gestion des requêtes asynchrones.

Création d'interfaces graphiques.



Pydantic permet de valider automatiquement les données reçues selon un modèle défini, évitant ainsi les erreurs liées aux données mal formatées.

Exercice

Associez les termes à leur description.

Framework Python pour créer des API rapides	Serveur ASGI pour exécuter FastAPI	Validation et structuration des données	Wrapper pour modèle IA local
FastAPI	Uvicorn	Pydantic	LLMClient



FastAPI est le framework, Uvicorn le serveur ASGI, Pydantic gère la validation des données, et LLMClient encapsule le modèle IA local.

Exercice

Ordonnez les étapes pour intégrer un modèle IA local dans le chatbot.

A Télécharger le modèle IA dans backend/models

B Créer un wrapper LLMClient pour le modèle

C Modifier l'endpoint /chat pour utiliser LLMClient

D Tester avec des requêtes curl

E Relancer le serveur backend



On commence par télécharger le modèle, créer le wrapper, modifier l'endpoint, relancer le serveur, puis tester les requêtes.

Exercice

Que fait la fonction `get_product_info` dans `services.py` ?

✓ Cherche un produit dans le dataset et retourne une description ou suggestion.

Génère une réponse IA pour toute requête.

Valide les données reçues du frontend.

Gère les requêtes HTTP.



`get_product_info` cherche un produit dans le dataset JSON et retourne soit une description soit une suggestion d'accord vin selon la requête.

Exercice

Le middleware CORS dans FastAPI permet d'autoriser les requêtes provenant uniquement des origines spécifiées. Vrai ou Faux ?

✓ Vrai

Faux



Le middleware CORS est configuré pour autoriser uniquement les origines listées, ce qui sécurise les échanges entre frontend et backend.

Exercice

Quels mots-clés déclenchent une escalade vers un humain dans le chatbot ?

✓ "pas"

"mauvais"

"bien"

"excellent"



Les mots "pas" et "mauvais" sont détectés comme indicateurs de sentiment négatif, déclenchant une escalade vers un humain.

[exercice p. 71] **Solution n°2**

La logique basée sur JSON est déterminée par des mots-clés précis appelés triggers.

Si le mot-clé est trouvé dans la requête de l'utilisateur, le chatbot retourne la réponse. Celle-ci correspond à la recommandation déterminée dans le fichier JSON.

Le JSON garantit une réponse fiable rapide et cohérente en se basant sur des produits connus.

L'avantage principal est la simplicité et la précision. Néanmoins, la limite est que le chatbot ne peut répondre qu'aux questions prévues dans le dataset. Si le mot clé n'existe pas, il ne peut rien proposer. C'est donc dans ce contexte qu'un fallback IA entre en jeu afin de couvrir les requêtes non couvertes par le JSON.

[exercice p. 71] **Solution n°3**

Utilisateur : "quel vin sans alcool pour accompagner un plateau de fromage charentais" ?

Premièrement, le chatbot reçoit le message.

Ensuite, il cherche dans la base JSON si le mot clé "fromage" existe comme trigger.

Si c'est le cas, le chatbot renvoie la réponse prévue.

Sinon, le chatbot se base sur le prompt pour générer une suggestion personnalisée puis il renvoie la réponse.

Étant donné que le mot-clé "vin sans alcool" n'est pas dans la base, le chatbot va demander au modèle IA de générer une suggestion adaptée.

Ainsi, le chatbot combine une base fiable JSON et une IA flexible.